

Final integrated implementation freeze - 17 Sep 2026

DriftSQL

Production-Oriented Implementation Blueprint

Six-Month Final-Year Research Prototype

Project	DriftSQL: An Enterprise Query Intelligence and Validation System
Central research component	EEVA - Enterprise Evidence Validation Algorithm
Team and duration	Two members Twenty-four weeks
Blueprint version	1.0
Date	16 August 2026
Status	Implementation-ready plan; dataset, license, schema, code and experimental results are NOT YET VERIFIED

Research position

This blueprint preserves the validated research architecture and converts it into a secure, testable, reproducible production-oriented research pilot. It does not claim commercial-scale production readiness or global novelty.

Six pre-implementation specifications are now resolved. Section 59 is authoritative for B5/B6, Gold R(q), unsupported queries, clarification authority, freshness rules and team ownership. Core architecture unchanged.

Document Control

Table 1. Document control and authority status.

Item	Value
Official file name	DriftSQL_Production_Implementation_Blueprint_6_Months
Research authority	DriftSQL Master Blueprint
Methodology authority	Prompt 2 Research Foundation
Literature authority	Prompt 1 Literature Foundation
Engineering reference	Production-Oriented 16-Week Blueprint
Latest instruction	
Dataset status	Olist source/license/files/schema verification pending
Source-code status	NOT YET VERIFIED
Measured results	NOT YET AVAILABLE

Table of Contents

No.	Section	Page
1	Executive Summary	5
2	Source Hierarchy and Evidence Discipline	5
3	Blueprint Conflict Register	5
4	Fixed Research Position	6
5	Scope and Priority Classification	6
6	Assumptions and Missing Inputs	6
7	Minimum Viable Research System	7
8	Final Logical Architecture	8
9	Stage-1 and Stage-2 Boundary	8
10	Runtime Data Flow	9
11	Offline Experimental Flow	10
12	Layer Classification	10
13	Legacy Module Implementation Mapping	10
14	Technology Stack and Alternatives	11
15	Repository Structure	11
16	PostgreSQL Data and Evidence Design	12
17	Shared Data Contracts	12
18	Source Authority and Conflict Policy	13
19	Enterprise Evidence Construction	14
20	Query Intelligence Design	14
21	AASE Implementation Inside QIM/Retrieval	14
22	Retrieval and Normalization Design	15
23	SDARE Implementation Inside Retrieval/Clarification	15
24	MAL Implementation Inside Evidence Normalization	15
25	EEVA Core Design	16
26	Evidence Decision Design	17
27	CARD and Clarification Recovery	18
28	Explainable Join Reasoning	18
29	Text-to-SQL Generation	19
30	SQL Validation and SFO Diagnostics	19
31	Bounded Self-Correction	19
32	Controlled Execution	19
33	Explainable Result	20
34	User Interface and APIs	20
35	Security	20
36	Audit and Observability	21
37	Testing Strategy	21
38	Dataset Audit and Acceptance	21
39	Question and EEVA Case Construction	22
40	Baselines	22
41	Ablations	22
42	Metrics	23
43	Experiment Execution Plan	23
44	Six-Month Timeline	24
45	24-Week Sprint Plan	24
46	Team Responsibilities	25
47	Risk Register	25
48	Simplification Path	26
49	CI/CD and Reproducibility	26
50	Backup and Recovery	26
51	Definition of Done	27
52	Demo Scenarios	27
53	Research-Paper Integration	27
54	Threats to Validity	28
55	Viva Explanation	28
56	First 14-Day Plan	28
57	Codex-Ready Backlog	30
58	Final Consistency Checklist	32
59	Implementation Freeze - Resolved Pre-Implementation Specifications	33

List of Figures

Figure	Page
Figure 1. Final DriftSQL logical architecture.	8
Figure 2. EEVA internal decision flow.	16
Figure 3. Bounded clarification and recovery loop.	18
Figure 4. Runtime data and control flow.	9
Figure 5. Offline experimental flow.	10
Figure 6. Production-oriented deployment architecture.	20
Figure 7. Six-month delivery timeline.	24

List of Tables

Table	Page
Table 1. Document control and authority status.	2
Table 2. Source hierarchy.	5
Table 3. Blueprint conflict register.	5
Table 4. Priority classification.	6
Table 5. Missing-input register.	6
Table 6. Minimum viable research system.	7
Table 7. Stage boundary.	8
Table 8. Layer classification.	10
Table 9. Legacy module implementation mapping.	10
Table 10. Technology stack and alternatives.	11
Table 11. Repository structure.	11
Table 12. Proposed PostgreSQL tables.	12
Table 13. Shared data contracts.	12
Table 14. Source authority policy.	13
Table 15. Enterprise evidence artifacts.	14
Table 16. QIM output fields.	14
Table 17. AASE implementation rules.	14
Table 18. Retrieval strategy.	15
Table 19. SDARE implementation rules.	15
Table 20. MAL implementation responsibilities.	15
Table 21. EEVA states.	16
Table 22. EEVA reason codes.	17
Table 23. Recovery actions.	18
Table 24. Stage-2 SFO diagnostic categories.	19
Table 25. API endpoints.	20
Table 26. Security controls.	20
Table 27. Audit fields and metrics.	21
Table 28. Testing matrix.	21
Table 29. Dataset acceptance criteria.	21
Table 30. Dataset targets.	22
Table 31. Baselines.	22
Table 32. Ablations.	22
Table 33. Metrics.	23
Table 34. Experiment scenarios.	23
Table 35. Six-month phase gates.	24
Table 36. Twenty-four week sprint plan.	24
Table 37. Team responsibilities.	25
Table 38. Risk register.	25
Table 39. Simplification path.	26
Table 40. CI/CD checks.	26
Table 41. Backup and recovery.	26
Table 42. Definition of done.	27
Table 43. Demo scenarios.	27
Table 44. Research-paper integration.	27
Table 45. Threats to validity.	28
Table 46. Viva answers.	28
Table 47. First fourteen-day plan.	28
Table 48. Codex-ready backlog.	30
Table 49. Final consistency checklist.	32

1. Executive Summary

DriftSQL is a production-oriented final-year research prototype for reliable enterprise Text-to-SQL. Its central research component, EEVA (Enterprise Evidence Validation Algorithm), validates query-specific enterprise evidence before an LLM may generate SQL. Evidence spans schema, metadata, approved business definitions, documentation, relationship/value facts, permission policy, authority, version, access scope and provenance. Olist Brazilian E-Commerce is the approved primary real-world database for implementation, development, Streamlit/live demonstration, enterprise evidence construction, the main case study and primary evaluation; its actual source, license, files and exact schema remain subject to verification.

The blueprint preserves one research architecture and one decisive causal experiment. QIM creates Required Evidence Obligations R(q); retrieval/normalization gathers typed evidence; EEVA returns GENERATE, CLARIFY or REJECT; validated evidence proceeds to explainable join reasoning and one frozen LLM; candidate SQL is checked independently by Stage-2 SQL Validation, receives at most two SQL-correction attempts when appropriate, and executes only through a controlled read-only PostgreSQL role. B5 and B6 keep Query Intelligence, retrieval, evidence sources, join reasoning, LLM/settings/prompt, Stage 2, correction and execution identical; only the Stage-1 EEVA gate differs.

One DriftSQL codebase supports database-specific access through database_id and generic database adapters. LiveSQLBench Base-Lite is a small secondary generalization evaluation only after the Olist implementation, EEVA rules and prompts are stable/frozen; its results are reported separately and never combined with Olist into one overall accuracy. EEVA remains a constrained rule-based decision procedure. Random Forest/XGBoost may be explored only as stretch soft support and may never override hard constraints. The plan retains typed contracts, tests, security, audit, CI/CD, reproducibility, monthly gates, the 24-week sprint plan and the Codex-ready backlog without adding multi-agent, Kubernetes or microservice complexity.

2. Source Hierarchy and Evidence Discipline

The blueprint resolves disagreements through an explicit authority order. Actual datasets, database catalogs and code control factual implementation details. The Master Blueprint controls the research identity and EEVA boundary. Prompt 2 controls the validated research gap and methodology. The production-oriented blueprint contributes engineering detail. The latest prompt controls the requested six-month format and priority system. General technical knowledge is used only for clearly labelled implementation recommendations.

Table 2. Source hierarchy.

Priority	Source	Controls
1	Latest explicit six-month prompt	Output files, document structure, 24-week schedule, priority labels.
2	Actual dataset, PostgreSQL schema and source code	Real fields, keys, relationships, implementation and test status.
3	DriftSQL Master Blueprint	Research identity, architecture, novelty boundary, RQs, baselines, B5/B6, metrics.
4	Prompt 2 Research Foundation	Validated gap, EEVA formalization, evidence states, experiments and risks.
5	Prompt 1 Literature Foundation	Paper-supported prior work and conservative base-paper positioning.
6	Production-Oriented 16-Week Blueprint	Stack, APIs, storage, security, observability, tests and deployment.
7	General technical knowledge	Alternatives only; must be labelled proposed.

Evidence discipline: no dataset schema, license, result, completed test, implementation status or performance value is invented. Any unsupported point is marked NOT YET VERIFIED and linked to a verification task.

3. Blueprint Conflict Register

Table 3. Blueprint conflict register.

ID	Topic	Difference	Type	Resolution
C-01	Schedule	Existing production plan uses 16 weeks; current requirement is six months.	Implementation conflict	Expand to 24 weeks while retaining its engineering controls.
C-02	Project title	Older Master version used a reliability subtitle.	Terminology conflict	Use the exact approved project title in both blueprints.

ID	Topic	Difference	Type	Resolution
C-03	Dataset size	Older plan fixed 120 questions; approved plan uses tiered targets.	Schedule conflict	Early 20/~100; minimum 60-80/~300-400; preferred ~100/~500; stretch up to 120 only if label quality remains high.
C-04	ML	Older production plan allows hybrid or rule-based EEVA.	Implementation conflict	Rule-based is mandatory; ML is stretch and cannot override hard constraints.
C-05	Vector store	Earlier material mentions ChromaDB; production plan recommends pgvector.	Implementation conflict	Keyword/simple index for early prototype; pgvector should-have; ChromaDB optional.
C-06	Legacy modules	Earlier acronyms appeared as separate concepts.	Architecture conflict	AASE, SDARE, MAL, SFO and CARD are merged; LCHR stretch; HF removed.
C-07	Production wording	Some drafts say production-level.	Research-scope conflict	Use production-oriented final-year research prototype, not commercial platform.
C-08	Correction/clarification bounds	One or two attempts appear in different plans.	Schedule conflict	Max 2 clarification cycles; max 2 SQL-correction attempts; stop earlier on no progress or hard failure.

4. Fixed Research Position

DriftSQL investigates whether a formal, query-conditioned validation of heterogeneous enterprise evidence before SQL generation can reduce false and unsafe Text-to-SQL generation under incomplete, stale, ambiguous, conflicting or unauthorized context while preserving acceptable utility and latency.

EEVA is not claimed as the first pre-generation validator, the first clarification system or the first permission-aware Text-to-SQL mechanism. Its scoped contribution is the formal integration of Required Evidence Obligations, typed multi-source evidence, source authority, non-compensatory hard constraints, recoverability and a reason-coded three-way decision before SQL generation.

- Primary research object: the query-specific enterprise evidence state.
- Primary decision: GENERATE, CLARIFY or REJECT.
- Primary safety metric: False Acceptance Rate.
- Primary utility metric: False Rejection Rate.
- Primary causal comparison: B5 without EEVA versus B6 with EEVA.
- Base paper relationship: adopt selective pre-generation control from Reliable Text-to-SQL with Adaptive Abstention, but validate a broader evidence object.

5. Scope and Priority Classification

Table 4. Priority classification.

Priority	Definition and examples
MUST HAVE	Olist primary PostgreSQL database; database_id + generic adapter; QIM/R(q); evidence repository/retrieval; constrained rule-based EEVA; reason codes; max two clarification cycles; one frozen LLM; join reasoning; SQLGlot Stage 2; max two corrections; read-only execution; UI; B5/B6; real metrics.
SHOULD HAVE	FastAPI; Docker Compose; audit trace; structured logs; automated tests/CI; retrieval metrics; p50/p95 latency; selected ablations; small LiveSQLBench Base-Lite secondary only after Olist freeze.
STRETCH	Random Forest/XGBoost soft support after the rule baseline; broader secondary benchmarks; advanced evidence graph; cloud/OpenTelemetry or extra dashboard work only if core research is complete.

6. Assumptions and Missing Inputs

Table 5. Missing-input register.

ID	Input	Status	Verification action
M-01	Actual Olist source files	NOT YET VERIFIED	Obtain/verify Olist files and

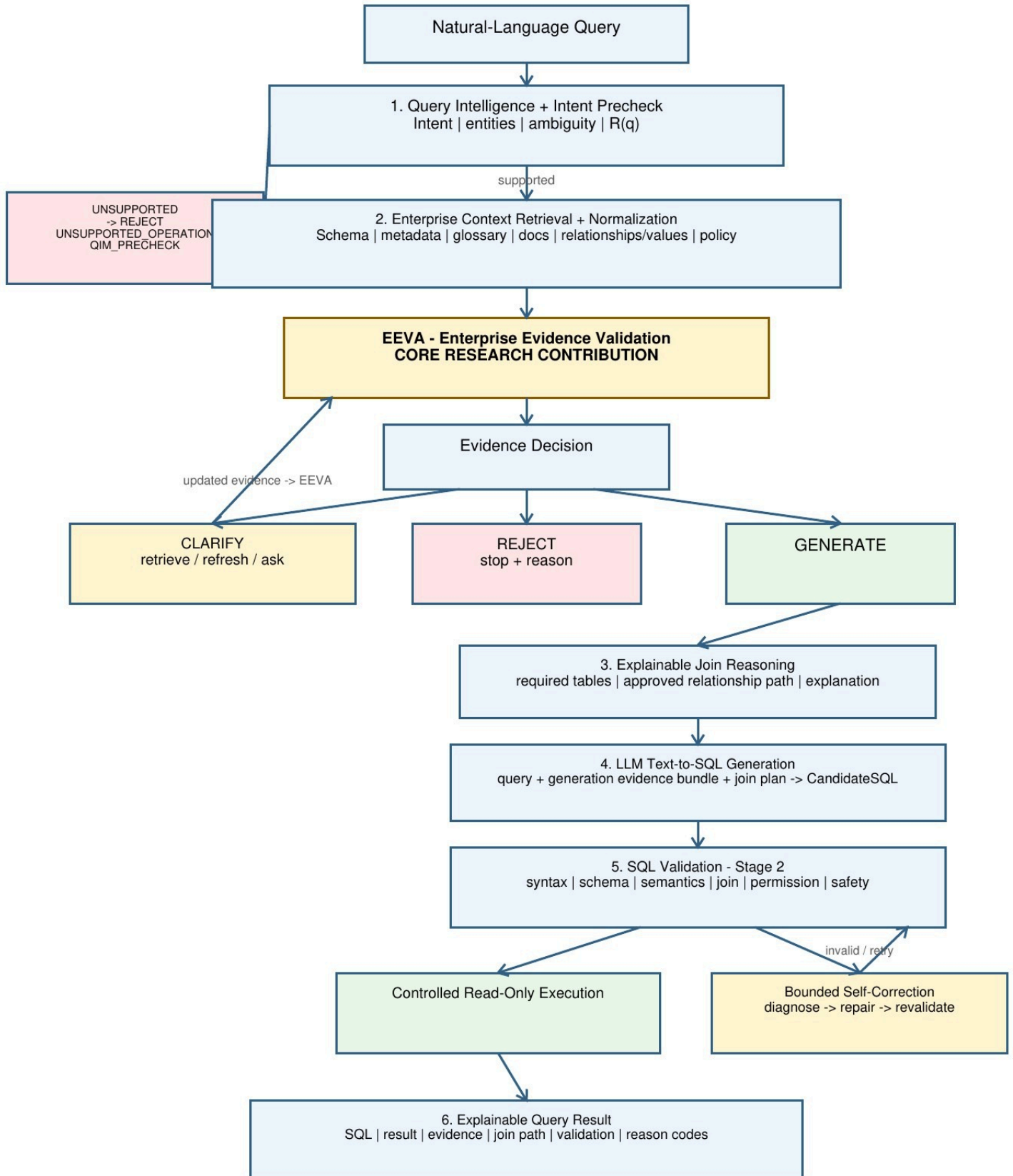
ID	Input	Status	Verification action
			verify source/license during DATA-001.
M-02	Olist source/license	NOT YET VERIFIED	Confirm exact source plus research/redistribution terms.
M-03	Actual relational schema	NOT YET VERIFIED	Profile files; do not invent tables or relationships.
M-04	Existing source repository	NOT YET VERIFIED	Upload repository or initialize only after approval.
M-05	PostgreSQL instance and credentials	NOT YET VERIFIED	Create local development instance during DATA-002.
M-06	Primary LLM and API budget	NOT YET VERIFIED	Select one model before GEN-001; freeze version/settings.
M-07	Embedding model	NOT YET VERIFIED	Select one after the evidence corpus size is known.
M-08	Faculty/domain label reviewer	NOT YET VERIFIED	Ask a reviewer to validate a sample of EEVA cases.
M-09	Measured results	NOT AVAILABLE	Replace paper placeholders only after EXP-003 and EXP-004.

7. Minimum Viable Research System

Table 6. Minimum viable research system.

Area	Minimum completion
Data	One audited Olist dataset in PostgreSQL; verified schema/policy sources; database_id registry and generic adapter interface.
Evidence	Schema, metadata, glossary, documentation, relationship/value facts and permission fixtures.
QIM	Structured QueryIntent and Required Evidence Obligations R(q).
EEVA	Rule-based state evaluation, hard gate, coverage, conflict logic, reason codes and three decisions.
Recovery	At most two clarification cycles; each must materially change, refresh or resolve evidence and re-run EEVA; no progress/exhaustion -> REJECT.
Generation	One frozen LLM and one versioned prompt.
Stage 2	AST parsing, SELECT/CTE allowlist, schema/join/policy checks, and at most two bounded SQL-correction attempts.
Execution	Read-only PostgreSQL role, timeout, row/result limit.
Evaluation	Primary B5 versus B6 on held-out Olist with actual Macro-F1, FAR, FRR, EX, valid-SQL rate and latency; Base-Lite reported separately after freeze.
Demonstration	Simple UI showing obligations, evidence, decisions, SQL, validation, trace and result.

8. Final Logical Architecture



QIM precheck handles unsupported operations before retrieval/EEVA. Supported analytical queries follow the unchanged EEVA-centered two-stage architecture.

Stage	Question	Input	Output	Must not do
Stage 1 - EEVA	Can the generator safely rely on the available enterprise evidence for this query?	Query, R(q), normalized evidence, role/policy	GENERATE / CLARIFY / REJECT + reasons	Generate or repair SQL.
Stage 2 - SQL Validation	Is the generated SQL structurally valid, schema-compatible, semantically appropriate, permission-compliant, safe, and executable?	Candidate SQL, validated bundle, join plan, catalog, role	PASS / FAIL + SQL diagnostics	Change the EEVA evidence decision.

A CLARIFY outcome returns updated evidence to EEVA. A REJECT outcome stops the pipeline before the generator. A Stage-2 failure may repair SQL only within the already validated evidence bundle.

10. Runtime Data Flow

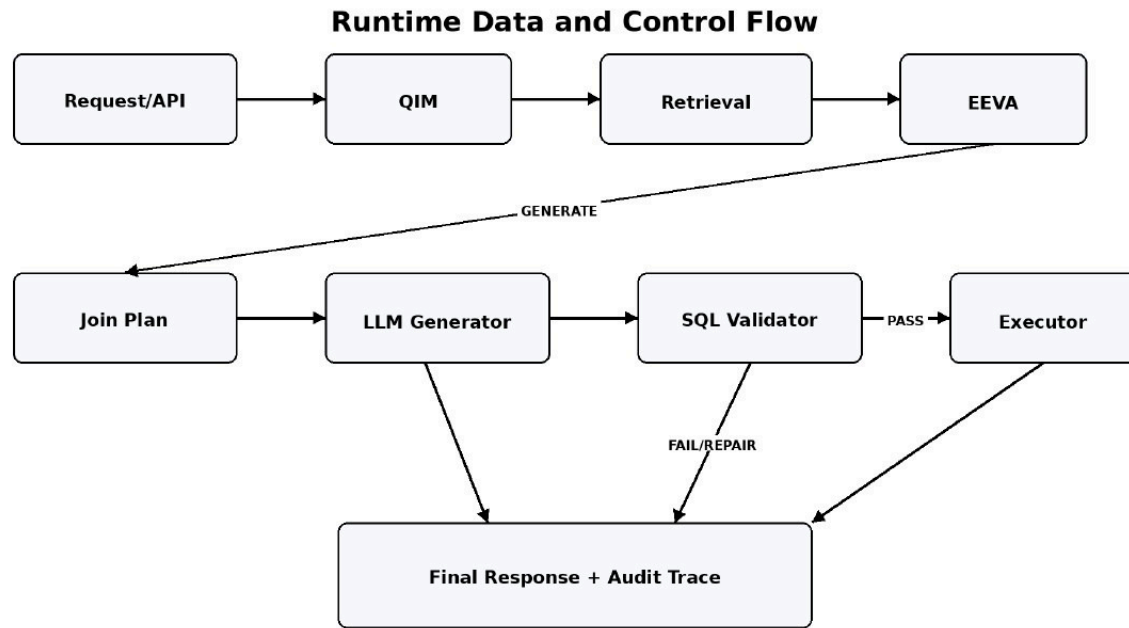


Figure 4. Runtime data and control flow.

1. Attach request ID, user identity, role, database_id and timestamp; select the registered generic database adapter.
2. QIM produces Query/Intent and R(q).
3. Direct and semantic retrieval use the active database adapter and evidence registry to produce a normalized EvidenceSnapshot.
4. EEVA evaluates states, hard failures, conflicts, mandatory coverage and recoverability.
5. CLARIFY updates evidence and returns to retrieval/normalization and EEVA (max 2, progress required); REJECT stops; GENERATE freezes a ValidatedEvidenceBundle.
6. Join reasoning selects approved relationship edges.
7. The LLM produces CandidateSQL from validated context only.
8. SQL Validation parses and checks the candidate; at most two repairs are allowed only for SQL-level defects.
9. Read-only execution returns a bounded result and an AuditTrace; PostgreSQL remains the final authorization authority.

11. Offline Experimental Flow

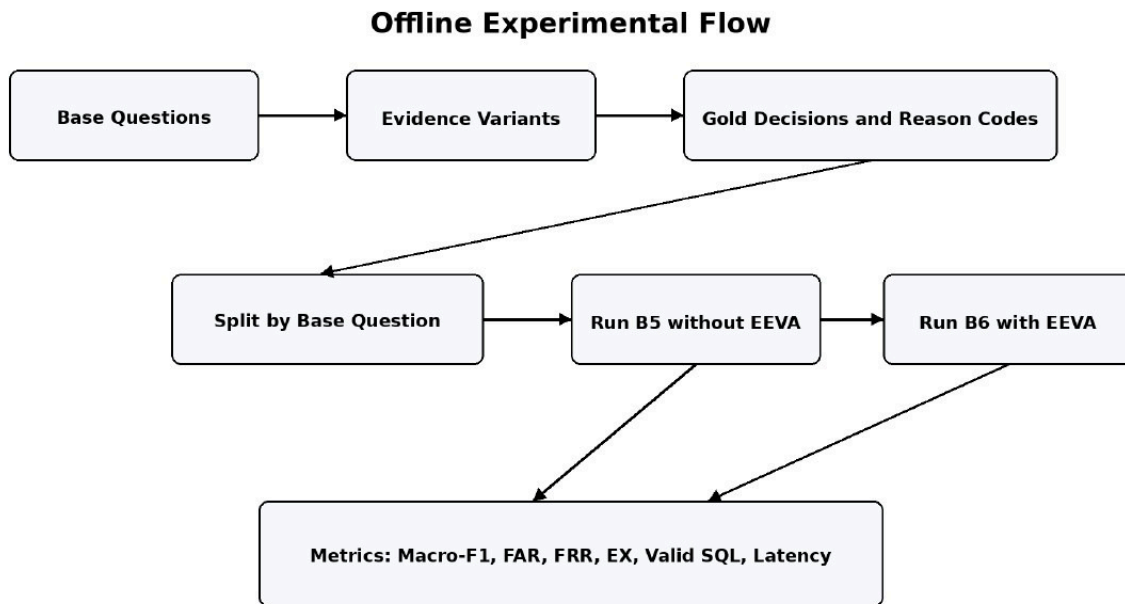


Figure 5. Offline experimental flow.

The offline pipeline builds a controlled benchmark without turning the evaluation questions into a runtime lookup list. All variants of one base question remain in the same split. B5 and B6 are run with identical downstream components, and the frozen held-out set is not changed after thresholds or rules are selected.

12. Layer Classification

Table 8. Layer classification.

Category	Layer	Responsibility	Priority
Runtime	Enterprise Data		MUST
Runtime	Evidence Repository	Versioned evidence items and snapshots.	MUST
Runtime	QIM	Intent and R(q).	MUST
Runtime	Retrieval/Normalization	Direct and semantic evidence acquisition.	MUST
Runtime	EEVA Core	States, hard gate, conflict, coverage, recoverability.	MUST
Runtime	Decision/Clarification	G/C/R and bounded recovery.	MUST
Runtime	Join/Generation	Approved relationship plan and one LLM.	MUST
Runtime	Stage 2/Execution	AST validation, correction, read-only execution.	MUST
Offline	Cases and experiments	Olist primary question/evidence cases plus later Base-Life secondary cases; B5/B6 and metrics remain separately reported.	MUST
Cross-cutting	Security, audit, tests, deployment	Engineering controls around all modules.	SHOULD

13. Legacy Module Implementation Mapping

Table 9. Legacy module implementation mapping.

Legacy	Status	Parent	Code location	Implementation	Priority	Contribution
EEVA	KEEP CORE	EEVA Core	src/eeva	Evidence states, rules, decisions	MUST	Yes - central

Legacy	Status	Parent	Code location	Implementation	Priority	Contribution
QIM	KEEP	Query Intelligence	src/ query_intelligence	Intent and R(q)	MUST	Supporting
AASE	MERGE	QIM + Retrieval	src/ query_intelligence; src/retrieval	Aliases, abbreviations, fuzzy/semantic mapping	MUST	No
SDARE	MERGE	Retrieval + Clarification	src/retrieval; src/clarification	Snapshot hash, drift detection, refresh	SHOULD	No
MAL	MERGE	Evidence Repository	src/evidence; src/contracts	Typed evidence abstraction	MUST	Part of C1
SFO	MERGE	SQL Validation	src/sql_validation; src/self_correction	Structured SQL failure diagnostics	MUST	No
CARD	MERGE	Clarification Controller	src/clarification; src/retrieval	Bounded retrieval expansion and recovery	MUST	Part of C2
LCHR	STRETCH	Model gateway	Future work	Local/cloud routing	STRETCH	No
HF	REMOVE	Covered by existing controls	None	No separate firewall	REMOVE	No
RAG	KEEP SUPPORTING	Retrieval	src/retrieval	Acquire evidence before EEVA	MUST	No
ChromaDB	OPTIONAL TOOL	Retrieval	Infrastructure choice	Vector index alternative	STRETCH	No
NetworkX	OPTIONAL TOOL	Join reasoning	src/join_reasoning	Graph path library	SHOULD	No

14. Technology Stack and Alternatives

Table 10. Technology stack and alternatives.

Technology	Use	Priority	Reason	Alternative
Python	All core modules and experiments	MUST	Single-language student stack	Java/TypeScript adds unnecessary complexity.
Pydantic	Typed contracts	MUST	Validation and JSON schemas	Dataclasses for earliest prototype.
PostgreSQL	Olist primary DB, catalogs, policy, execution	MUST	Authoritative schema, roles, transactions	MySQL possible but less aligned to plan.
psycopg / SQLAlchemy	DB access	MUST	Controlled connections and metadata	Direct psycopg for simpler prototype.
Alembic	Migrations	SHOULD	Versioned schema changes	Manual SQL migrations early.
FastAPI	Typed service API	SHOULD	Clear contracts and health endpoints	Streamlit-only mandatory demo.
Streamlit	Reviewer-facing UI	MUST	Fast, transparent demo	Simple web form.
SQLGlott	AST SQL validation	MUST	Structural safety checks	Equivalent PostgreSQL parser.
pgvector	Semantic retrieval	SHOULD	One database service	FAISS / in-memory index / ChromaDB.
Sentence Transformers	Embeddings	SHOULD	Local and reproducible	Hosted embedding, one frozen model.
pytest	Tests	MUST	Unit and integration coverage	unittest.
Docker Compose	Reproducibility	SHOULD	Student-manageable deployment	Documented local setup.
One frozen LLM	Generation and optional QIM assist	MUST	Keeps EEVA as research variable	Local SQL model if budget/privacy requires.

15. Repository Structure

Table 11. Repository structure.

Path	Purpose	Priority
apps/api	FastAPI endpoint and endpoints	SHOULD
apps/ui	Streamlit demonstration UI	MUST
src/contracts	Pydantic contracts and enums	MUST
src/data	Olist loaders/profiles + database registry/adapters	MUST
src/evidence	Evidence repository, snapshots and provenance	MUST
src/query_intelligence	QIM and AASE functionality	MUST
src/retrieval	Direct/semantic retrieval and normalization	MUST
src/eeva	Core EEVA rules and decisions	MUST
src/clarification	CARD-style bounded recovery	MUST

Path	Purpose	Priority
src/join_reasoning	Relationship graph and JoinPlan	MUST
src/text_to_sql	LLM gateway and prompt versions	MUST
src/sql_validation	AST and SFO diagnostics	MUST
src/self_correction	Bounded SQL repair	MUST
src/execution	Read-only executor	MUST
src/explanation	FinalResponse construction	MUST
src/audit	AuditTrace and logs	SHOULD
experiments	Baselines, ablations, metrics, results	MUST
tests	Unit, integration, golden, adversarial, security	MUST/SHOULD
docs	Audits, decisions, setup and viva	MUST
docker and workflows	Containers and CI	SHOULD

```

driftsql/
  apps/api/          apps/ui/
  src/contracts/     src/data/          src/evidence/
  src/query_intelligence/ src/retrieval/  src/eeva/
  src/clarification/  src/join_reasoning/
  src/text_to_sql/    src/sql_validation/
  src/self_correction/ src/execution/    src/explanation/
  src/audit/          src/security/
  datasets/raw/        datasets/processed/ datasets/held_out/
  experiments/baselines/ experiments/ablations/ experiments/results/
  tests/unit/          tests/integration/ tests/golden/
  docs/                scripts/          migrations/
  docker-compose.yml   pyproject.toml   README.md

```

16. PostgreSQL Data and Evidence Design

Table 12. Proposed PostgreSQL tables.

Table/group	Purpose	Priority
schema_snapshots	Immutable catalog snapshot keyed by database_id: snapshot ID, hash, created_at	MUST
schema_tables / schema_columns	Current structural facts and descriptions	MUST
schema_relationships	PK/FK and approved semantic edges	MUST
metadata_entries	Descriptions, units, aliases, sensitivity, version	MUST
glossary_terms	Approved terms and definitions with effective dates	MUST
documents / document_versions	Versioned business rules and source authority	MUST
permission_policies	Role-to-table/column/operation policy fixtures	MUST
evidence_items / evidence_snapshots	Normalized typed evidence and request snapshots	MUST
query_requests / required_evidence_obligations	Runtime request and R(q)	MUST
eeva_decisions / recovery_actions	Decision, reason codes, cycles and outcomes	MUST
sql_candidates / sql_validation_results	Generated SQL and Stage-2 diagnostics	MUST
execution_results / audit_events	Bounded results and trace	SHOULD
experiment_runs / experiment_cases / metric_results	Frozen experimental records	MUST
users / roles / user_roles	Application authentication fixtures	SHOULD

Implementation may begin with version-controlled JSON for glossary, documentation and experiment cases, then migrate to PostgreSQL tables after contracts stabilize. Every request, schema snapshot, evidence snapshot and experiment case carries database_id. Register Olist first; activate a LiveSQLBench Base-Lite adapter only after the Olist implementation and rules/prompts are frozen. The normal Olist transactional data remains unchanged; controlled faults belong in evidence artifacts or controlled schema-drift copies. Business data and authoritative permissions must never be fabricated by the LLM.

17. Shared Data Contracts

Table 13. Shared data contracts.

Contract	Key fields	Flow
QueryRequest	request_id, user_id, role, query, database_id, timestamp, clarification_state	API/UI -> QIM
QueryIntent	intent, entities, terms, metrics, filters, time, ranking, ambiguity, scope_precheck	QIM -> retrieval/controller
RequiredEvidenceObligation	id, type, canonical_target_id/key, mandatory, sources, constraint, recovery, reason	QIM -> retrieval/EEVA
EvidenceItem	id, type, key/value, source/version/effective dates, authority, access, provenance, hash	Repository/retrieval -> EEVA
EvidenceSnapshot	snapshot_id, request_id, item IDs, schema hash, retrieval metadata, freshness_policy_version	Retrieval -> EEVA/audit
EEVAResult	decision, states, coverage, hard failures, conflicts, reasons, recovery/bundle ID	EEVA -> controller
RecoveryAction	type, target source, expected change, attempt, timeout, status	EEVA -> clarification
GenerationEvidenceBundle	common downstream interface; BaselineGenerationBundle (B5) or ValidatedEvidenceBundle (B6)	Controller -> join/generator
JoinPlan	tables, edges, path, explanation, relationship versions	Join -> generator/validator
SQLCandidate	SQL, model/prompt versions, bundle ID, join-plan ID	Generator -> Stage 2
SQLValidationResult	PASS/FAIL and structured diagnostics	Validator -> executor/correction
ExecutionResult	status, rows, time, truncation, safe error	Executor -> result
FinalResponse	decision, decision_stage, reason_codes, result/SQL, evidence/join/status/trace ID	Application -> UI
AuditTrace	full lifecycle + versions incl. freshness_policy_version and bundle/snapshot IDs	All modules -> audit
ExperimentCase / MetricResult	initial_snapshot_id, recovery_snapshot_ids, gold_rq_version, frozen config, measured outputs	Experiments

Example EvidenceItem

```
{
  "evidence_id": "EV-001",
  "database_id": "olist_primary",
  "type": "GLOSSARY",
  "key": "verified business term",
  "value": "NOT YET VERIFIED - define only from verified Olist semantics",
  "source": "approved glossary",
  "source_version": "v1",
  "effective_from": "YYYY-MM-DD",
  "effective_to": null,
  "retrieved_at": "timestamp",
  "authority_level": 90,
  "access_label": "internal",
  "scope": "olist_primary_database",
  "provenance_id": "PROV-001",
  "content_hash": "sha256...",
  "linked_requirement_ids": ["REQ-001"]
}
```

18. Source Authority and Conflict Policy

Table 14. Source authority policy.

Domain	Primary authority	Secondary authority	Low-authority support
Structural schema	Live PostgreSQL catalog	Approved current snapshot	Cached descriptions; examples
Authorization	PostgreSQL role/policy	Application fixture	Prompt instruction
Business semantics	Approved current glossary	Current approved documentation	Curated metadata; examples
Relationships	Live PK/FK catalog	Approved relationship documentation	Inferred/example relationships
Value formats	Bounded read-only probe	Approved metadata	Examples

A conflict is recoverable when version/effective date or a clearly higher-authority source can resolve it through an approved action. Freshness is source-specific: live catalog for schema/relationships, active DB policy for authorization, latest applicable approved glossary/docs, and current approved metadata snapshot. No arbitrary global age threshold is used.

permitted recovery. Evidence with invalid provenance is not used unless replaced by an approved source. The LLM never selects authority.

19. Enterprise Evidence Construction

Table 15. Enterprise evidence artifacts.

Artifact	Source	Construction	Authority	Versioning
Schema snapshot	PostgreSQL catalog	Automatic	Highest for structure	Hash and timestamp each approved snapshot
Metadata catalog	Curated descriptions plus profiling	Mixed	High when approved	Version descriptions, units, aliases, date semantics
Business glossary	Faculty/team approved definitions	Manual	Highest for business meaning	Effective dates and approval status
Documentation store	Short business rules and formulas	Manual/versioned	Secondary to glossary	Source ID, authority and effective period
Relationship facts	PK/FK catalog and approved notes	Automatic/mixed	Highest from live catalog	Record edge provenance and version
Value facts	Non-sensitive samples/aggregates	Bounded probe	Supporting	Minimize raw rows; capture format only
Permission policies	PostgreSQL roles and fixtures	Manual/configured	Highest for access	Role, table/column, allowed operations
Gold obligations	Reviewed oracle per evaluation question	Manual	Evaluation-only	Never feed as hidden runtime hints

20. Query Intelligence Design

Table 16. QIM output fields.

Output field	Meaning
Intent	lookup, aggregate, rank, compare, trend, existence or unsupported
Entities	business entities and values
Metrics	measure and aggregation
Filters	categorical and numerical conditions
Temporal conditions	dates, periods and relative expressions
Ranking	top/bottom and limits
Ambiguity	unresolved user meaning or competing interpretations
Sensitive resources	potential restricted tables/columns
R(q)	mandatory/optional evidence obligations with reasons

Implementation approach: deterministic parsing for stable patterns and one structured-output LLM call only when needed. First run the read-only scope precheck; unsupported write/DDL/admin requests return final REJECT / UNSUPPORTED_OPERATION at QIM_PRECHECK with no generator/executor call. Supported queries produce QueryIntent and R(q). One manually reviewed Gold R(q) per base question is stored offline for QIM metrics only and is never visible to runtime.

21. AASE Implementation Inside QIM/Retrieval

Table 17. AASE implementation rules.

Function	Implementation rule
Alias registry	Map approved aliases and abbreviations to canonical schema/business terms.
Fuzzy matching	Use bounded similarity only as a candidate signal, never authority.
Semantic matching	Use one embedding model for terms, metadata and schema descriptions.
Disambiguation	Return multiple candidates and CLARIFY when no authoritative resolution exists.
Provenance	Every mapping records source and confidence.
Testing	Typos, abbreviations, synonyms, ambiguous aliases and false-positive controls.

22. Retrieval and Normalization Design

Table 18. Retrieval strategy.

Evidence	Method	Control	Priority
Schema	Direct authoritative lookup	No semantic-only authority	MUST
Permissions	Direct role/policy lookup	Fail closed if unavailable	MUST
Metadata	Exact + keyword + semantic	Preserve versions/aliases/units	MUST
Glossary	Exact term + semantic top-k	Return competing definitions; do not collapse conflicts	MUST
Documentation	Chunk and semantic retrieve	Preserve source/effective date/authority	MUST
Relationships	Catalog graph + approved notes	Safe probes only when an obligation needs confirmation	MUST
Examples	Optional low-authority context	Cannot override mandatory evidence	STRETCH

Normalization converts all results into `EvidenceItem` records, deduplicates exact duplicates, links items to obligations, preserves contradictory records and records retrieval rank. Retrieval quality is measured separately from EEVA using `Recall@K`, `NDCG@K` and missing-obligation retrieval rate.

23. SDARE Implementation Inside Retrieval/Clarification

Table 19. SDARE implementation rules.

Capability	Rule
Schema fingerprint	Hash normalized table, column, type, PK and FK catalog.
Drift detection	Compare the active catalog with the evidence snapshot.
State	Mark outdated cached items <code>STALE_RECOVERABLE</code> when live refresh exists.
Refresh	Create a new immutable schema snapshot and refresh dependent metadata/index entries.
No silent mutation	The current request keeps its snapshot; a recovery cycle explicitly adopts the new one.
Test cases	Renamed column, removed column, changed type, changed relationship and no-op refresh.

24. MAL Implementation Inside Evidence Normalization

Table 20. MAL implementation responsibilities.

Responsibility	Implementation
Typed abstraction	Represent schema, metadata, glossary, docs, relationships, values and policies uniformly.
Access filtering	Remove evidence the active role may not retrieve.
Version control	Every item references version/effective period and snapshot.
Provenance	Record source and content hash.
Conflict preservation	Never deduplicate away contradictory claims.
Output	<code>EvidenceSnapshot</code> suitable for EEVA and audit.

25. EEVA Core Design

EEVA Internal Decision Flow

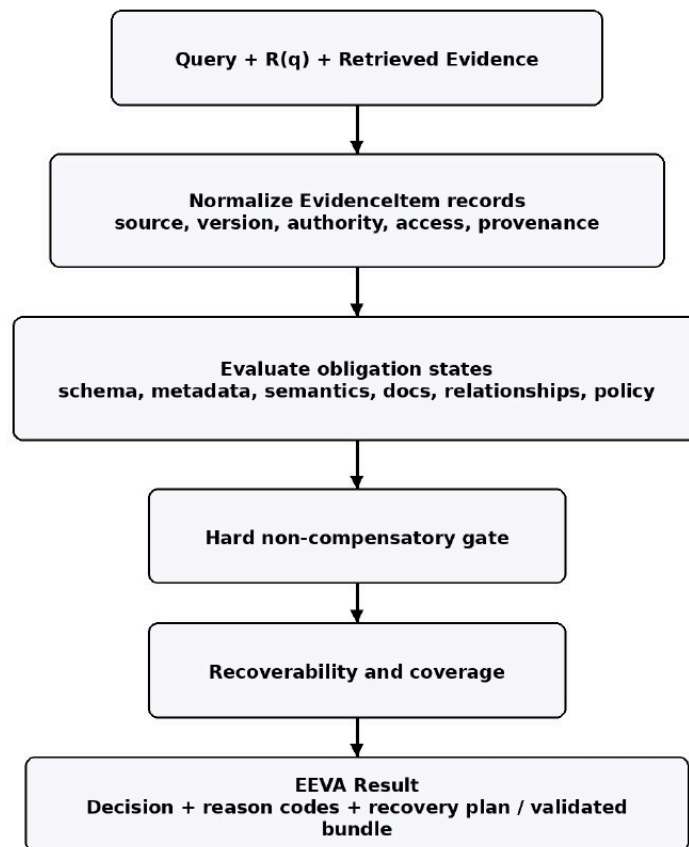


Figure 2. EEVA internal decision flow.

Table 21. EEVA states.

State	Meaning	Decision effect
PASS	Mandatory evidence is current, consistent, authorized and sufficiently supported.	Generate candidate allowed if all mandatory obligations pass.
MISSING_RECOVERABLE	Required evidence absent from current retrieval but an approved source exists.	CLARIFY.
STALE_RECOVERABLE	Evidence outdated but a live/current source can refresh it.	CLARIFY.
AMBIGUOUS_RECOVERABLE	More than one plausible meaning can be resolved.	CLARIFY.
CONFLICT_RECOVERABLE	Sources disagree but authority/version recovery is possible.	CLARIFY.
MISSING_CRITICAL	Mandatory evidence confirmed absent; no approved recovery.	REJECT.
CONFLICT_CRITICAL	Authoritative contradiction remains unresolved.	REJECT.
UNAUTHORIZED	Mandatory evidence or operation is denied.	REJECT.
INVALID	Integrity or provenance failure with no trusted replacement.	REJECT.

Coverage and decision logic

$C_m(q,E)$ = number of mandatory obligations in PASS / total mandatory obligations

$C_o(q,E)$ = weighted optional obligations satisfied / total optional weight

No value of C_o may override UNAUTHORIZED, MISSING_CRITICAL, CONFLICT_CRITICAL or INVALID.

if any hard failure: REJECT

elif any recoverable state: CLARIFY

elif $C_m == 1$ and all mandatory obligations PASS: GENERATE

```
else: CLARIFY if an approved recovery remains, otherwise REJECT
```

26. Evidence Decision Design

Table 22. EEVA reason codes.

Code	Meaning
SCHEMA_MISSING_CRITICAL	Mandatory structural item absent.
SCHEMA_STALE_RECOVERABLE	Cached schema differs from live catalog.
METADATA_MISSING_RECOVERABLE	Needed meaning/unit/alias can be retrieved.
METADATA_INVALID	Metadata integrity or source invalid.
GLOSSARY_AMBIGUOUS	Multiple approved/plausible meanings.
DOCUMENT_CONFLICT_RECOVERABLE	Version/authority may resolve conflict.
DOCUMENT_CONFLICT_CRITICAL	Authoritative conflict unresolved.
RELATIONSHIP_MISSING_CRITICAL	No valid required join path.
RELATIONSHIP_STALE_RECOVERABLE	Cached edge differs from current catalog.
PERMISSION_DENIED	Active role lacks required access.
PROVENANCE_INVALID	Evidence source cannot be trusted.
RETRIEVAL_COVERAGE_LOW	Required items not sufficiently retrieved.
RECOVERY_NO_PROGRESS	Recovery produced no evidence change.
RECOVERY_EXHAUSTED	Bound reached without resolution.
ALL_MANDATORY_EVIDENCE_PASS	Generate path approved.

EEVAResult example

```
{
  "decision": "CLARIFY",
  "mandatory_coverage": 0.8,
  "optional_coverage": 0.5,
  "evidence_states": [
    {
      "requirement_id": "REQ-004",
      "state": "GLOSSARY_AMBIGUOUS"
    }
  ],
  "hard_failures": [],
  "conflicts": [
    "revenue_definition"
  ],
  "missing_items": [],
  "recovery_actions": [
    "ASK_USER_OR_RETRIEVE_CURRENT_GLOSSARY"
  ],
  "provenance_summary": [
    "GLOSSARY-v2",
    "DOC-v1"
  ],
  "reason_codes": [
    "GLOSSARY_AMBIGUOUS"
  ],
  "clarification_cycle": 1,
  "validated_evidence_bundle_id": null
}
```

27. CARD and Clarification Recovery

Bounded Clarification and Recovery Loop

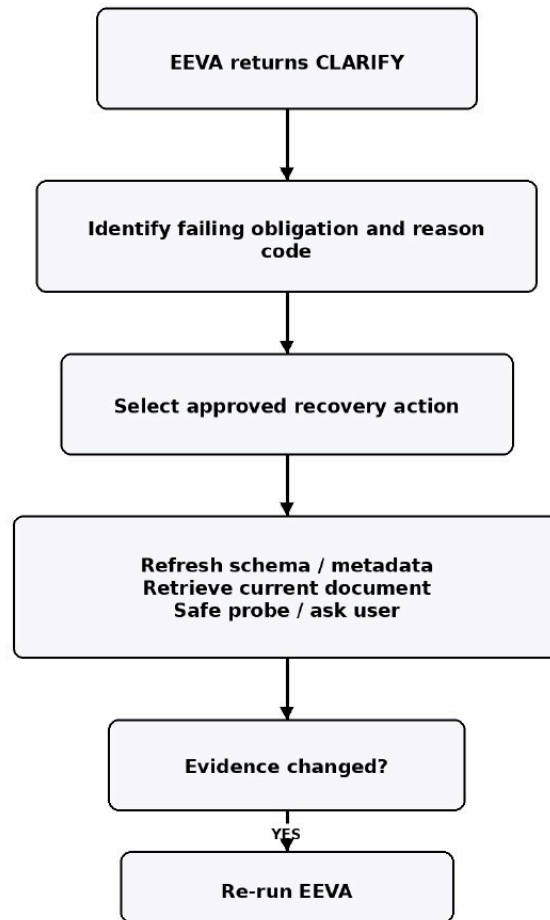


Figure 3. Bounded clarification and recovery loop.

Table 23. Recovery actions.

Reason/state	Approved recovery
SCHEMA_STALE_RECOVERABLE	Refresh live catalog and create a new snapshot.
METADATA_MISSING_RECOVERABLE	Retrieve approved metadata or profile non-sensitive fields.
GLOSSARY_AMBIGUOUS	Ask a focused user question or retrieve the current approved definition.
DOCUMENT_CONFLICT_RECOVERABLE	Retrieve the latest effective document and re-evaluate authority.
RELATIONSHIP_STALE_RECOVERABLE	Refresh PK/FK graph or run a safe catalog probe.

Clarification authority boundary: a user may resolve intent, filters or choose among approved interpretations, but cannot create or override schema/relationship facts, DB permissions/policy, provenance/source authority or approved enterprise definitions. Accepted clarification updates QueryIntent/R(q), rebuilds/updates EvidenceSnapshot through approved recovery actions, and MUST re-enter EEVA. Max two cycles; no progress/exhaustion -> REJECT / RECOVERY_EXHAUSTED.

28. Explainable Join Reasoning

Join reasoning runs only after GENERATE. It builds a graph from authoritative relationship evidence, connects the tables required by R(q), selects a valid path, records PK/FK edges and produces a concise path explanation. When no valid path exists, the result is not silently repaired with LLM inference; the evidence should have caused CLARIFY or REJECT before generation.

- Input: ValidatedEvidenceBundle and QueryIntent.
- Output: JoinPlan with tables, edges, path, relationship versions and explanation.
- Implementation: adjacency list is sufficient; NetworkX is optional.
- Tests: direct relationship, multi-hop path, ambiguous alternatives, missing path and stale edge.

29. Text-to-SQL Generation

Use one frozen instruction-following LLM for the primary experiment. The prompt includes the original question, approved schema subset, approved business definitions, validated evidence IDs, JoinPlan, PostgreSQL dialect, SELECT-only policy and output constraints. The model name, version, parameters, prompt template and retry policy are versioned in SQLCandidate and AuditTrace.

SYSTEM: Produce one PostgreSQL SELECT or safe WITH...SELECT statement.
 CONTEXT: only the validated schema, definitions and join plan below are authoritative.
 DO NOT: invent tables/columns, use DDL/DML, or include multiple statements.
 OUTPUT: JSON {sql, selected_tables, selected_columns, assumptions}.

30. SQL Validation and SFO Diagnostics

Stage 2 parses CandidateSQL into an AST, enforces an allowlist, validates identifiers and joins against approved evidence, re-checks permissions and produces structured diagnostics. String matching is not the primary safety mechanism.

Table 24. Stage-2 SFO diagnostic categories.

Diagnostic	Meaning
PARSE_ERROR	SQL cannot be parsed.
STATEMENT_NOT_ALLOWED	DDL/DML or multiple statements detected.
TABLE_NOT_FOUND	Referenced table absent from authoritative catalog.
COLUMN_NOT_FOUND	Referenced column absent.
JOIN_MISMATCH	Join edge differs from JoinPlan or approved relationship.
AGGREGATION_MISMATCH	Required grouping/aggregation absent or inconsistent.
FILTER_MISMATCH	Required filter/time/ranking not represented where checkable.
PERMISSION_FAILURE	Referenced object denied.
PREFLIGHT_FAILURE	EXPLAIN or read-only preflight failed.

31. Bounded Self-Correction

Self-correction repairs only diagnosed SQL-level defects and uses the same validated evidence bundle and JoinPlan. The runtime permits at most two SQL-correction attempts. After every repair, run the full Stage-2 SQL Validation again. Stop early on repeated diagnostics, no AST progress, unsafe output, permission failure, use of unapproved evidence, or when the configured bound is reached.

```
for attempt in range(max_attempts):
    repaired = generator.repair(candidate, validation.diagnostics,
                                validated_bundle, join_plan)
    validation = validate(repaired)
    if validation.pass: return repaired
    if no_progress(previous_ast, repaired_ast) or repeated_failure: break
return correction_failure
```

32. Controlled Execution

Only Stage-2 PASS SQL executes. Use a dedicated read-only PostgreSQL role, read-only transaction, statement timeout, row/result-size limit, restricted schema access, safe errors and rollback or safe close. PostgreSQL is the final access-control authority even when EEVA has already checked policy evidence.

- No DDL or DML.
- No multiple statements.
- No automatic retry after permission denial or unsafe failure.
- Record execution status, rows, latency, timeout and truncation in ExecutionResult and AuditTrace.

33. Explainable Result

The user-facing response should expose enough information to audit the outcome without revealing private chain-of-thought or unauthorized data. GENERATE responses contain SQL, bounded result, EEVA decision, evidence summary, selected tables/columns, join path, Stage-2 status, correction count and trace ID. CLARIFY responses contain the unresolved obligation and focused recovery request. REJECT responses contain safe reason codes and confirm that generation and execution did not occur.

34. User Interface and APIs

Production-Oriented Deployment Architecture

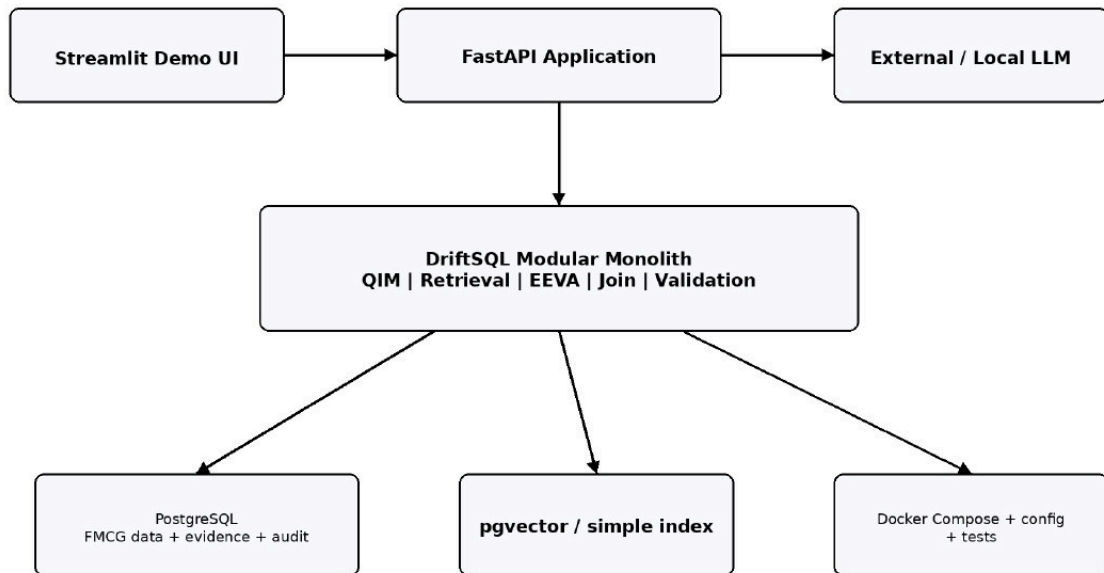


Figure 6. Production-oriented deployment architecture.

Table 25. API endpoints.

Endpoint	Contract	Priority	Behavior
POST /query	QueryRequest -> FinalResponse	MUST via UI; API SHOULD	Create request, run pipeline, return decision/result.
POST /clarify	request_id + user answer -> FinalResponse	SHOULD	Update evidence and re-run EEVA.
GET /requests/{id}	Request status and summary	SHOULD	Reviewer trace access.
GET /requests/{id}/audit	AuditTrace	SHOULD	Restricted to authorized reviewer/admin.
POST /admin/evidence/refresh	Source/snapshot request	STRETCH	Create a new evidence snapshot.
GET /health	Health response	SHOULD	DB/model dependency status.
GET /metrics	Operational metrics	STRETCH	Admin-only metrics.

The mandatory interface may be Streamlit-only. It must display QueryIntent, R(q), evidence items, EEVA states, decision/reasons, join plan, candidate SQL, validation diagnostics, correction attempts, result and trace ID.

35. Security

Table 26. Security controls.

Control	Implementation	Priority
Least privilege	Dedicated application and execution roles; read-only SQL role.	MUST
Secrets	Environment variables; never commit API/DB secrets.	MUST

Control	Implementation	Priority
Evidence access filtering	Retrieve only evidence allowed for active role.	MUST
Prompt injection	Treat documents as data; strip/flag instructions; prompts define authority externally.	SHOULD
SQL injection/unsafe SQL	AST allowlist and parameterization where values are supplied.	MUST
Sensitive values	Use non-sensitive samples/aggregates only.	MUST
Safe errors	Do not expose credentials, stack traces or restricted metadata.	MUST
Input limits	Question and document-size limits.	SHOULD
Audit integrity	Append-only or restricted audit records.	SHOULD
Dependency scanning	Basic CI vulnerability scan.	SHOULD
Backups	DB dump, evidence snapshot and versioned configuration.	SHOULD

36. Audit and Observability

Table 27. Audit fields and metrics.

Area	Recorded fields
Identity	request_id, trace_id, user_id, role, database_id
QIM	version, QueryIntent, obligation IDs
Evidence	snapshot ID, item IDs, versions, ranks, authority
EEVA	version/rule set, states, coverage, decision, reason codes, recovery
Generation	model, prompt version, latency, candidate hash
Stage 2	diagnostics, correction attempts, final SQL hash
Execution	latency, rows, timeout, truncation, status
Outcome	FinalResponse type and error category

Mandatory observability is a structured AuditTrace and per-module timing. p50/p95 latency and health endpoints are should-have. OpenTelemetry is stretch; a PostgreSQL audit table plus structured JSON logs is sufficient for the pilot.

37. Testing Strategy

Table 28. Testing matrix.

Suite	Coverage	Priority
Unit	QIM extraction; normalization; source authority; each hard rule; recoverable states; coverage; reason codes; AST parser; permission checks; termination.	MUST
Integration	Clean -> GENERATE; recoverable -> CLARIFY -> EEVA; critical/permission -> REJECT; REJECT never calls LLM; invalid SQL -> correction; valid SQL -> executor.	MUST
Golden	Stable small question set with expected decision and result.	MUST
Adversarial	Prompt injection, poisoned document, fake authority, stale versions, unauthorized columns, multi-statement and DDL/DML.	SHOULD
Regression	Previously fixed cases remain fixed.	SHOULD
Experiment validation	Split leakage checks; B5/B6 config diff; frozen test set.	MUST

38. Dataset Audit and Acceptance

Table 29. Dataset acceptance criteria.

Criterion	Acceptance evidence
Source/license	Source and research-use status documented.
Files	All raw files inventoried.
Profile	Rows, columns, types, nulls, duplicates and examples measured.

Criterion	Acceptance evidence
Entities	Identify e-commerce entities only from actual Olist files. Customers/orders/items/products/sellers/payments/reviews/location may be considered only when verified; never invent names or links.
Keys	Candidate primary and foreign keys validated from data.
Relationships	Enough meaningful relations for join questions.
Business semantics	Verified Olist semantics support glossary/documentation cases.
Question capacity	Supports at least the minimum 60-80 reviewed questions.
Decision	Suitable / load-adjustment required / BLOCKED until a verification issue is resolved; do not substitute unrelated data by default.

Olist is the approved primary database, but its actual source, license, files and exact schema still require verification before implementation facts are asserted. DATA-001 remains the first data task. The normal Olist transactional data is kept unchanged for the primary case study; controlled faults are introduced only through evidence snapshots/artifacts or controlled schema-drift copies. No table, column, key or relationship in this document is an assertion until verified from the actual files/catalog.

39. Question and EEVA Case Construction

Table 30. Dataset targets.

Tier	Base questions	Evidence cases	Use
Early demo	20	~100	Review/demo only
Minimum final	60-80	300-400	Acceptable if carefully reviewed
Preferred final	~100	~500	Recommended
Stretch	Up to 120	Not fixed; expand only if annotation quality remains high.	Only if label quality remains high

Question categories: simple lookup, filters, aggregation, ranking, joins, temporal queries, business-semantic queries, ambiguity, permissions and unsupported requests. Each case stores the base question ID, evidence snapshot, expected states/decision/reason codes/recovery, and verified SQL or expected result where needed. Gold/reference SQL, expected results and oracle labels are offline evaluation data only and are never supplied to runtime as hidden hints. Split by base question, never by variant. After Olist rules/prompts are frozen, run a small LiveSQLBench Base-Lite subset through the same codebase using database_id/generic adapters; report its results separately and never compute a combined Olist + LiveSQLBench overall accuracy.

40. Baselines

Table 31. Baselines.

ID	System	Method	Priority	Purpose
B0	Direct LLM Text-to-SQL	Question + broad schema -> LLM -> SQL	MUST	Reference direct prompting.
B1	RAG Text-to-SQL	Retrieve top-k context and pass directly to LLM	MUST	Separates retrieval from validation.
B2	Schema-aware Text-to-SQL	Schema filter/retriever before generation	SHOULD	Tests schema-only gains.
B3	Clarify/abstain baseline	Ambiguity/uncertainty clarifies without multi-source EEVA	SHOULD	Tests whether three-way behavior alone is enough.
B4	Permission-aware baseline	Policy allow/refuse around request/SQL	STRETCH	Tests permission-only safety.
B5	DriftSQL without EEVA	Initial snapshot -> BaselineGenerationBundle -> same downstream	MUST	Primary causal baseline.
B6	DriftSQL with EEVA	Initial snapshot -> EEVA -> ValidatedEvidenceBundle (GENERATE)	MUST	CLARIFY may create recovery snapshots.

Frozen pair: same INITIAL EvidenceSnapshot; B6 recovery snapshots are treatment outputs. Join reasoning must not re-implement EEVA checks.

41. Ablations

Table 32. Ablations.

Ablation	Change	Priority
Full EEVA	All dimensions active	MUST reference
Without EEVA	Remove Stage 1 only	MUST full held-out set
Schema-only EEVA	Validate only schema obligations	SHOULD stress subset
No conflict detection	Disable cross-source conflicts	SHOULD stress subset
No recoverability	Collapse CLARIFY to REJECT	SHOULD stress subset
No early permission gate	Keep Stage 2/DB enforcement	STRETCH
Rule-only versus hybrid ML	Compare soft classifier only after rule baseline	STRETCH

42. Metrics

Table 33. Metrics.

Layer	Metric	Reason
QIM	Mandatory Obligation Recall	Primary Gold R(q) metric.
QIM	Obligation Precision / F1	Canonical obligation quality.
QIM	Unsupported Intent Accuracy	Read-only scope precheck metric.
EEVA	Macro-F1 + class P/R/F1	G/C/R decision quality.
EEVA	FAR / FRR	Reliability / utility trade-off.
EEVA	Conflict F1	Cross-source conflict detection.
EEVA	Permission / false permit	Safety diagnostic.
EEVA	Clarification recovery	Recoverable cases restored.
EEVA	Reason-code accuracy	Explanation matches oracle.
Retrieval	Recall@K / NDCG@K	Retrieval quality upstream.
SQL	Execution Accuracy	Primary downstream semantic metric.
SQL	Valid SQL / Error Rate	Stage-2 diagnostics.
SQL	Correction success	Repair efficiency.
System	Latency + loop counts	p50/p95 and bounded cycles.

43. Experiment Execution Plan

Table 34. Experiment scenarios.

Experiment	Setup	Baselines	Metrics	What it tests
E1 Normal evidence	Clean held-out evidence	B0/B1/B5/B6	EX, valid SQL, FRR, latency	EEVA does not over-block.
E2 Missing evidence	Remove required retrievable or critical item	B1/B3/B5/B6	Macro-F1, FAR, recovery, EX	Completeness and recovery.
E3 Schema drift	Stale snapshot versus live catalog	B2/B5/B6	Drift detection, FAR, recovery EX	Freshness handling.
E4 Metadata inconsistency	Missing/stale/contradictory description or unit	B1/B5/B6	State accuracy, FAR, EX	Value beyond schema-only.
E5 Business ambiguity	Competing metric definitions	B3/B5/B6	Clarify recall, FAR, post-clarification EX	Semantic clarification.
E6 Documentation conflict	Contradictory approved-looking sources	B1/B5/B6	Conflict P/R/F1, FAR	Cross-source contribution.
E7 Permission violation	Role denied mandatory object	B4/B5/B6	Detection, false permit, generator-called flag	Early safety integration.
E8 Multiple conflicts	Combine stale/conflict/permission/noise	B5/B6	Macro-F1, FAR, reason accuracy	Non-compensatory logic.
E9 EEVA ablation	Same INITIAL EvidenceSnapshot; paired B5 vs B6	B5/B6	FAR, FRR, EX, latency	Causal contribution.
E10 End-to-end	Mixed clean and corrupted cases	B0-B6	EX, valid SQL, unsafe generation, correction load	System trade-off.

Primary experiments E1-E10 are run on Olist. The small LiveSQLBench Base-Lite subset is a separate Month-6 generalization evaluation after Olist freeze; results remain separate and are not combined into one overall accuracy.

44. Six-Month Timeline

Six-Month Delivery Timeline

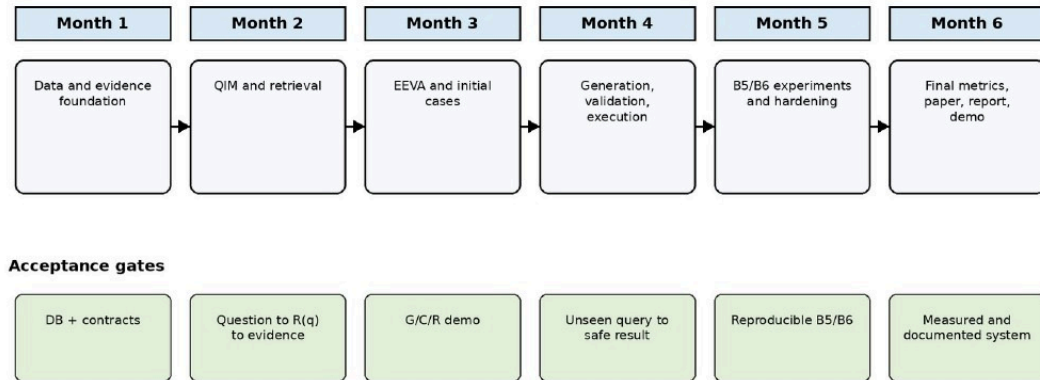


Figure 7. Six-month delivery timeline.

Table 35. Six-month phase gates.

Phase	Focus	Acceptance gate
Month 1	Olist data/evidence foundation + database_id/adapters	Olist source/license/files audited; repeatable PostgreSQL load; real keys/relationships validated; schema snapshot/hash; olist_primary registered.
Month 2	Evidence repository, QIM/R(q), retrieval/normalization	Initial 20 questions/-100 variants; query -> R(q) -> normalized evidence works.
Month 3	EEVA states, hard constraints, conflict/authority, recovery	Working G/C/R; max-two clarification/no-progress/RECOVERY_EXHAUSTED tests; controlled scenarios.
Month 4	Join, one LLM, Stage 2, max-two correction, execution/UI	Unseen Olist question -> controlled, explainable, read-only result.
Month 5	Olist B0-B6 as feasible; B5/B6; ablations; freeze	Primary Olist B5/B6 reproducible; preliminary measured metrics; EEVA rules/prompts frozen.
Month 6	Final Olist + Base-Lite secondary; paper/demo	Final Olist metrics plus separately reported LiveSQLBench Base-Lite results; no combined score; final package.

45. 24-Week Sprint Plan

Table 36. Twenty-four week sprint plan.

Week	Task IDs	Owner	Focus	Deliverable	Acceptance	Risk	Fallback
1	DATA-001	Both	Verify Olist source/license; inventory all files; profile rows/columns/types/nulls/duplicates	docs/dataset_audit.md	Measured Olist profile; no invented schema	Source/file gap	Mark BLOCKED; obtain verified Olist materials
2	DATA-002	Member 1	PostgreSQL setup + schema-preserving Olist import	scripts/import_olist.py; import SQL	Repeatable import; row counts match; raw Olist unchanged	Import/type failure	Use staging/types without changing source data
3	DATA-003	Both	Validate real keys/relationships; register database_id; define generic adapter interface	docs/data_model.md; src/data/adapters	Verified relations; olist_primary registry/adapters tests	Unverified relation	Block relation-dependent work; no invented link
4	EVID-001/002	Member 1	Contracts + automatic schema extraction/hash keyed by database_id	src/contracts; schema snapshot	Contract tests; catalog snapshot/hash matches Olist DB	Contract churn	Freeze v1 and log changes
5	EVID-003/004	Both	Metadata and glossary v1	evidence files/tables	Versioned approved records	Subjective terms	Reviewer sample
6	EVID-005/006	Member 2	Documentation and permissions	docs/policies	Conflict and denial fixtures	Incomplete policy	Use minimal roles
7	QIM-001/002	Member 2	Query contracts and R(q)	src/query_intelligence	Structured output tests	LLM variance	Deterministic fallback
8	QIM-003/AASE	Member 2	Parser, aliases and ambiguity	QIM tests	20 questions produce obligations	False mappings	CLARIFY candidates
9	RET-001	Member 2	Direct retrieval	src/retrieval	Schema/policy retrieval exact	Missing source	Fail closed
10	RET-002/MAL	Both	Semantic retrieval and	EvidenceSnapshot	Recall@K on initial set	Vector delay	Keyword retrieval

Week	Task IDs	Owner	Focus	Deliverable	Acceptance	Risk	Fallback
			normalization				
11	SDARE	Member 2	Snapshot drift and refresh	drift tests	Renamed/changed schema detected	No live DB	Fixture snapshots
12	EEVA-001/002	Member 1	States and hard gate	src/eeva	Hard-rule unit tests	Rule ambiguity	Decision log
13	EEVA-003/004	Both	Coverage and conflicts	EEVA tests	Conflict states match oracle	Subjective conflicts	Authority rules
14	EEVA-005/006	Member 1	Reason codes and decision engine	EEVAResult	20-question G/C/R demo	Over-reject	Tune on dev only
15	EEVA-007/008	Member 2	Recovery controller	src/clarification	Max-two-cycle recovery; no-progress and RECOVERY_EXHAUSTED tests	Looping	Strict bound
16	JOIN-001	Member 2	Join plan	src/join_reasoning	Approved paths explained	Ambiguous path	CLARIFY/REJECT
17	GEN-001	Member 2	One LLM and prompt	src/text_to_sql	Versioned candidate output	Cost/API	Local or cached dev
18	VAL-001/002	Member 1	AST and schema validation	src/sql_validation	DDL/DML blocked; IDs validated	Parser mismatch	PostgreSQL dialect tests
19	VAL-003/CORR	Both	Policy validation and max-two SQL repair	src/self_correction	Revalidation bounded to <=2 correction attempts	Unsafe repair	Stop and report
20	EXEC/UI	Both	Read-only execution and UI	src/execution; apps/ui	End-to-end unseen query	DB role issue	Local read-only fixture
21	EXP-001/002	Both	Expand to minimum 50-80/-300-400 or preferred ~100/-500; freeze base-question splits	datasets/held_out	Tier target met; variants grouped by base question; leakage checks pass	Label load	Use minimum tier
22	EXP-003	Both	Run primary held-out Olist B5/B6 and required evidence scenarios	experiment outputs	Reproducible configs and raw metrics	No EEVA gain	Error analysis; narrow claim
23	EXP-004/Hardening	Both	Final Olist metrics, tests, Docker/audit hardening, freeze EEVA rules/prompts	results; docker	Measured Olist charts + release candidate; secondary gate opened only after freeze	Time pressure	Drop stretch work
24	Submission	Both	LiveSQLBench Base-Lite secondary evaluation; paper, report, demo and viva	separate secondary results; final package	Base-Lite results separate from Olist; no combined score; DoD complete	Last-minute defects	Frozen demo backup

46. Team Responsibilities

Table 37. Team responsibilities.

Area	Primary owner	Shared review
PostgreSQL, schema, migrations	Member 1	Member 2 reviews data semantics
Evidence repository, metadata/glossary	Member 1	Both approve contracts and definitions
QIM and AASE	Member 2	Member 1 reviews R(q) schema needs
Retrieval, MAL, SDARE	Member 2 primary	Member 1 reviews authoritative sources
EEVA hard rules and decision engine	Member 1	Both review labels and errors
Conflict detection and clarification	Member 2	Both review recovery behavior
Join reasoning and LLM gateway	Member 2	Member 1 reviews SQL safety interfaces
SQL validation, SFO and executor	Member 1	Member 2 reviews generated cases
Experiments, graphs and UI	Member 2 primary	Both own B5/B6 and results
Paper, report, demo and viva	Shared	Both must explain complete system

47. Risk Register

Table 38. Risk register.

ID	Risk	Probability	Impact	Early sign	Prevention	Owner	Fallback
R1	Olist source/schema verification issue	Medium	High	Missing/unverified source, keys of relationships	Audit actual Olist materials in week 1; schema/relationship decisions only from evidence	Both	Block affected tasks and obtain verified Olist materials; do not invent/substitute unrelated data
R2	Olist source/license unclear	Medium	High	No source/license file	Verify exact source/license before publication or redistribution	Member 1	Use a legally usable verified Olist source or pause redistribution claims
R3	Subjective evidence labels	High	High	Frequent disagreement	Authority policy; dual review; faculty sample	Both	Reduce scope to clear cases
R4	Train/test leakage	Medium	High	Variants of same question across splits	Group split by base question; automated check	Member 2	Rebuild splits before experiments
R5	EEVA collapses to schema checks	Medium	High	No glossary/document conflict cases	Mandatory heterogeneous cases and schema-only ablation	Both	Strengthen evidence layer before scaling

ID	Risk	Probability	Impact	Early sign	Prevention	Owner	Fallback
R6	No measurable EEVA gain	Medium	High	FAR unchanged or FRR excessive	Early 100-case B5/B6 checkpoint	Both	Error analysis; narrow claim; adjust dev rules only
R7	LLM instability/cost	Medium	Medium	Frequent output variation or budget pressure	Freeze model/settings; cache dev responses	Member 2	Use local/cheaper SQL model
R8	Unsafe SQL	Low with controls	Critical	Validator misses DDL/DML/multi-statement	AST allowlist and security tests	Member 1	Fail closed; no execution
R9	Clarification latency/loops	Medium	Medium	Repeated no-progress cycle	Two-cycle cap with progress hash	Member 2	Convert to REJECT with reason
R10	Scope growth	High	High	Stretch tasks before B5/B6	Priority gate and weekly demo	Both	Apply simplification path
R11	Integration problems	Medium	High	Long-lived branches and incompatible contracts	Small PRs, weekly integration, versioned contracts	Both	Freeze interfaces and revert
R12	Novelty overclaim	Medium	High	Use of first/unique language	Paper checklist and conservative wording	Both	Rewrite claims to investigated integration

48. Simplification Path

Table 39. Simplification path.

Action	Items
Preserve first	Olist primary; database_id/generic adapter; QIM/R(q); simple retrieval; constrained rule EEVA; G/C/R; max two clarification cycles; simple joins; one LLM; AST validation; max two corrections; read-only execution; B5/B6; core metrics.
Postpone first	Random Forest/XGBoost; broader secondary benchmarks; advanced evidence graph/NetworkX complexity; FastAPI admin features; OpenTelemetry/cloud; UI polish.
Never remove	Stage boundary; reason codes; heterogeneous evidence; database_id/adapter discipline; split-by-base-question; B5/B6; real metrics; read-only PostgreSQL safety.

49. CI/CD and Reproducibility

Table 40. CI/CD checks.

Check	Implementation	Priority
Formatting/lint	ruff or equivalent	SHOULD
Type checks	mypy/pyright where practical	SHOULD
Unit tests	pytest	MUST
Integration tests	PostgreSQL test container/fixture	MUST
Migration validation	Alembic upgrade on clean DB	SHOULD
Security tests	DDL/DML, permissions, prompt-injection fixtures	MUST/SHOULD
Docker build	API/UI/DB Compose	SHOULD
Smoke test	Golden query after build	SHOULD
Experiment reproducibility	Frozen config, seed, model/prompt/evidence versions	MUST

50. Backup and Recovery

Table 41. Backup and recovery.

Asset	Recovery approach
Business/evidence database	Versioned PostgreSQL dump before final experiments and demo.
Evidence snapshot	Immutable snapshot IDs and exported JSON.
Configuration	Versioned rule, prompt, model and threshold files.
Application	Versioned Git tag and Docker image where feasible.
Rollback	Restore previous DB dump, snapshot and tested image.
Demo fallback	Local seeded environment and backup screen recording.

51. Definition of Done

No.	Completion requirement
1	Unseen question travels through the full supported pipeline.
2	QIM produces validated QueryIntent and R(q).
3	Evidence carries source, version, authority, access and provenance.
4	EEVA returns stable decisions and reason codes.
5	Hard failures cannot be overridden by optional evidence.
6	EEVA REJECT never calls the generator.
7	CLARIFY changes evidence or intent and returns through retrieval/normalization to EEVA.
8	Unsupported write/DDL/admin operations stop at QIM_PRECHECK with generator_called=false and execution_called=false.
9	GENERATE freezes a ValidatedEvidenceBundle for the production/B6 path.
10	Join reasoning uses the common GenerationEvidenceBundle interface and does not re-implement EEVA checks.
11	The LLM receives only the bundle supplied by the controller plus the approved JoinPlan.
12	SQL is parsed structurally with an AST.
13	DDL/DML/multiple statements are blocked.
14	Stage 1 and Stage 2 remain separate.
15	Self-correction is bounded to at most two attempts and re-runs the full Stage-2 validator after each repair.
16	Execution is read-only with timeout and result limits.
17	Every request has an AuditTrace including snapshot/bundle IDs and policy versions.
18	Required unit, integration, golden and experiment-validation tests pass.
19	Data splits are grouped by base question.
20	Paired B5/B6 runs share the same INITIAL EvidenceSnapshot; only eeava_enabled differs initially.
21	Real metrics are calculated from frozen outputs.
22	Paper placeholders are replaced only with measured values.
23	The system is reproducible from documentation and frozen configuration.
24	Demo shows Generate, Clarify, Reject, unsupported-precheck and bounded correction behavior.
25	No unsupported novelty claim appears.

52. Demo Scenarios

Scenario	Input condition	Required demonstration
D1 Clean Generate	Unseen clean question	Show R(q), evidence PASS, GENERATE, join plan, SQL, validation and result.
D2 Clarify and recover	Stale/missing recoverable evidence	Show reason code, recovery, new snapshot, EEVA re-run and generation.
D3 Reject permission	Denied table/column	Show REJECT and prove generator/executor not called.
D4 Critical conflict	Current authoritative rules disagree	Show conflict reason and safe stop.
D5 SQL self-correction	Controlled minor SQL defect	Show Stage-2 diagnostic, max-two repair and full revalidation.
D6 Unsupported precheck	INSERT/UPDATE/DDDL request	Show QIM_PRECHECK / UNSUPPORTED_OPERATION and no generator/executor call.
D7 Research result	B5 versus B6 chart	Explain FAR/FRR/EX/latency using actual measured values only.

53. Research-Paper Integration

Paper area	Implementation evidence
Problem formulation	EvidenceItem, R(q), states, source authority/currentness, QIM precheck and two-stage boundary.
Method	Architecture, EEVA decision policy, recovery authority, common generation-bundle interface and Stage-2 validator.
Experimental setup	Olist primary evidence/splits + controlled scenarios/B0-B6; same INITIAL EvidenceSnapshot for paired B5/B6; Base-Lite secondary only after freeze.
Results	B5/B6, scenario breakdown, QIM obligation metrics, ablations, latency and error analysis using measured values only.
Discussion	Safety-utility trade-off; Olist external-validity limit; small Base-Lite secondary; DBMS boundary.

Paper area	Implementation evidence
	DBMS boundary.
Required revision	Replace every illustrative value with measured results and label uncertainty honestly.
Allowed claim	Integrated query-conditioned enterprise evidence-validation layer; no global first claim.

54. Threats to Validity

Table 45. Threats to validity.

Threat	Risk	Mitigation
Construct validity	EEVA labels may reflect the rule design.	Independent review and reason-code oracle; report inter-review disagreement.
Internal validity	B5/B6 may differ in more than EEVA.	Shared code/config; automated configuration diff.
External validity	Olist is one primary real-world domain; the Base-Lite subset is intentionally small.	Report Olist and LiveSQLBench Base-Lite separately; do not overgeneralize; broader benchmarks remain future work.
Ecological validity	Controlled corruptions may be unrealistic.	Document generation procedure and review plausibility.
Annotation validity	Glossary/document definitions are partly curated.	Version and authority records; reviewer sample.
Model dependence	One LLM version may change behavior.	Freeze model/prompt/settings and record versions.
Security validity	EEVA is not a final security boundary.	PostgreSQL roles and read-only execution remain authoritative.
Statistical validity	Small held-out set may produce unstable estimates.	Report counts, confidence intervals where feasible and per-scenario results.

55. Viva Explanation

Table 46. Viva answers.

Question	Answer
What is DriftSQL?	An enterprise Text-to-SQL research prototype that validates query-specific enterprise evidence before SQL generation.
What is EEVA?	A Stage-1 evidence-state validator producing GENERATE, CLARIFY or REJECT with reason codes.
Why is EEVA different from SQL validation?	EEVA runs before SQL exists; SQL Validation checks the candidate afterward.
What is actually proposed?	Query-conditioned enterprise evidence validation: R(q), typed multi-source evidence, authority/version/provenance, hard constraints, recoverability, and reason-coded G/C/R before SQL.
Why use ML?	Only as optional soft decision support after a rule baseline; never for hard safety overrides.
Why Olist?	Real anonymized multi-table e-commerce data for PostgreSQL, evidence construction, joins and live demo; exact Olist schema is verified, not assumed.
What proves EEVA matters?	The identical-pipeline B5 versus B6 experiment using FAR, FRR, Macro-F1, EX and latency.
Why production-oriented?	Versioned evidence, typed contracts, read-only safety, audits, tests and reproducible deployment - not commercial scale.

56. First 14-Day Plan

Table 47. First fourteen-day plan.

Day	Owner	Task	Output	Acceptance	Risk	Fallback
1	Both	Inventory project files and all actual Olist source files	File inventory and source register	All files listed; missing items explicit	Missing Olist files/source	Mark DATA-001 BLOCKED; obtain verified Olist materials
2	Member 1	Verify Olist source/license evidence	docs/dataset_source.md	Status: verified/unverified/missing	License/source unclear	Clarify legal source before publication/redistribution
3	Member 2	Profile Olist rows, columns and data types	artifacts/dataset_profile.json	Counts measured	Large/dirty file	Chunked profile
4	Both	Measure Olist nulls, duplicates and sample formats	docs/dataset_audit.md	Quality table complete	Encoding issues	Normalize encoding copy

Day	Owner	Task	Output	Acceptance	Risk	Fallback
5	Both	Identify candidate entities/stables only from actual Olist files	docs/ data_model_draft.md	Candidates linked to actual fields	Flat file	Propose normalization
6	Member 1	Validate candidate Olist keys and relationships from data/catalog	docs/ key_validation.md	Candidates labelled confirmed/rejected	No stable keys	Surrogate keys with caveat
7	Both	Approve Olist data-audit outcome	docs/ dataset_decisions.md	VERIFIED / LOAD-ADJUST / BLOCKED	Unverified source/schema issue	Block affected tasks; do not replace with unrelated data
8	Member 1	Set up local PostgreSQL and staging schema	docker-compose or setup doc	Connection and staging table work	Docker issue	Native PostgreSQL
9	Member 1	Create repeatable Olist PostgreSQL import	scripts/ import_raw.py / SQL	Row counts match profile	Type failures	Load text staging first
10	Both	Finalize schema-preserving load/type plan and migration order; keep source Olist data unchanged	migrations/plan.md	Reviewed schema proposal	Relationship uncertainty	Keep raw/staging data and document any load-time type normalization
11	Member 2	Define QueryRequest and QueryIntent contracts	src/contracts/ query.py	Pydantic tests pass	Field churn	Version v1 and log changes
12	Member 1	Define EvidenceItem and EvidenceSnapshot	src/contracts/ evidence.py	Validation tests pass	Too many fields	Keep required core + optional metadata
13	Both	Define RequiredEvidenceObligation and EEVAResult	src/contracts/ eeva.py	Example JSON validates	Decision ambiguity	Use fixed state taxonomy
14	Both	Repository/test setup and two-week review	README, pytest config, decision log	Clean clone runs contract tests	Time loss	Defer Docker, keep local setup

57. Codex-Ready Backlog

Table 48. Codex-ready backlog.

ID	Priority	Title	Objective	Files	Inputs	Acceptance/tests	Dependencies	Effort	Owner
DATA-001	MUST	Olist audit	Verify source/license/files; profile actual Olist data and suitability	docs/dataset_audit.md; artifacts/dataset_profile.js on	Actual Olist source files	Measured counts/quality; candidate entities/keys; no invented schema	None	1-2 days	Both
DATA-002	MUST	Olist PostgreSQL import	Repeatable schema-preserving Olist import	scripts/import_olist.py; migrations/import.sql	Approved DATA-001	Row counts match profile; rerunnable; source data unchanged	DATA-001	2 days	Member 1
DATA-003	MUST	Relational validation + adapter	Validate Olist keys/relations; register database_id; define generic adapter	docs/data_model.md; src/data/adapters/	Raw import	Real keys/relations verified; olist_primary registered; adapter tests pass; no changed transactional data	DATA-002	3-5 days	Both
EVID-001	MUST	EvidenceItem contract	Freeze v1 evidence contract	src/contracts/evidence.py; tests/unit/test_evidence_contract.py	Data decisions	Pydantic examples validate	DATA-001	1 day	Member 1
EVID-002	MUST	Schema extraction	Create database_id-keyed catalog snapshot/hash	src/evidence/schema.py	PostgreSQL core schema	Snapshot/hash matches active Olist catalog	DATA-003	2 days	Member 1
EVID-003	MUST	Metadata catalog	Create descriptions, units, aliases	evidence/metadata.json; loader	Schema snapshot	Version/provenance tests	EVID-002	3 days	Both
EVID-004	MUST	Glossary store	Versioned approved business terms	evidence/glossary.json	Dataset semantics	Authority/effective-date validation	EVID-003	3 days	Both
EVID-005	MUST	Documentation store	Versioned business rules and conflict fixtures	evidence/documents/	Glossary	Retrieval and conflict tests	EVID-004	3 days	Member 2
EVID-006	MUST	Permission fixtures	Create role/table/column policies	migrations/permissions.sql; evidence/policies.json	Core schema	Denied case enforced by PostgreSQL	DATA-003	2 days	Member 1
QIM-001	MUST	QueryRequest contract	Request identity and role schema	src/contracts/query.py	Contracts setup	Validation tests	None	0.5 day	Member 2
QIM-002	MUST	R(q) contract	RequiredEvidenceObligation model	src/contracts/obligation.py	QIM-001	Mandatory/optional/ source/recovery fields validate	QIM-001	1 day	Member 2
QIM-003	MUST	Query parser	Produce QueryIntent and obligations	src/query_intelligence/parser.py	20 demo questions	Expected intent/obligation tests	QIM-002	4 days	Member 2
AASE-001	MUST	Alias resolution	Resolve abbreviations/synonyms with provenance	src/query_intelligence/aliases.py	Metadata aliases	Ambiguity does not silently resolve	EVID-003,QIM-003	2 days	Member 2
RET-001	MUST	Direct retrieval	Schema/policy exact retrieval	src/retrieval/direct.py	Evidence stores	Authoritative lookup tests	EVID-002,EVID-006	2 days	Member 2
RET-002	MUST	Semantic retrieval	Glossary/docs/metadata ranking	src/retrieval/semantic.py	Embedding/index choice	Recall@K on demo set	EVID-003-005	3 days	Member 2
RET-003	MUST	Normalization	Build immutable EvidenceSnapshot	src/retrieval/normalize.py	RET-001/002	Conflicts preserved; source/version attached	RET-001,RET-002	2 days	Both
SDARE-001	SHOULD	Schema drift detection	Compare snapshot hash to live catalog	src/retrieval/drift.py	EVID-002	Renamed/changed fixtures detected	EVID-002	2 days	Member 2
EEVA-001	MUST	Evidence-state enum	Implement fixed state taxonomy	src/eeva/states.py	EVID-001	Enum and serialization tests	EVID-001	0.5 day	Member 1
EEVA-002	MUST	Hard-constraint engine	Fail closed on permission/critical schema/conflict/provenance	src/eeva/hard_rules.py	Policies and evidence	Every hard rule unit-tested	EEVA-001	3 days	Member 1
EEVA-003	MUST	Coverage calculator	Compute C_m and C_o	src/eeva/coverage.py	R(q), states	Formula tests incl. zero/optional cases	QIM-002,EEVA-001	1 day	Member 1
EEVA-004	MUST	Conflict detector	Apply source authority and classify conflicts	src/eeva/conflicts.py	Glossary/docs/metadata	Recoverable/critical fixtures pass	EVID-004-005	3 days	Both
EEVA-005	MUST	Reason-code engine	Produce stable reason codes	src/eeva/reasons.py	States/conflicts	Reason-code oracle tests	EEVA-002-004	1 day	Member 1
EEVA-006	MUST	Decision engine	Return G/C/R and bundle/recovery	src/eeva/decision.py	All EEVA modules	20-question demo matches oracle	EEVA-002-005	2 days	Member 1
EEVA-007	MUST	Recovery planner	Map recoverable states to approved actions	src/clarification/planner.py	EEVAResult	Action tests and policy controls	EEVA-006	2 days	Member 2
EEVA-008	MUST	Cycle control	Bound clarification and detect no progress	src/clarification/controller.py	Recovery planner	Max-two-cycle behavior; no-progress and RECOVERY_EXHAUSTED tests	EEVA-007	2 days	Member 2
JOIN-001	MUST	Relationship graph	Produce validated JoinPlan	src/join_reasoning/graph.py	Schema relationships	Path/missing/ambiguous tests	EVID-002,EEVA-006	3 days	Member 2

ID	Priority	Title	Objective	Files	Inputs	Acceptance/tests	Dependencies	Effort	Owner
GEN-001	MUST	LLM prompt contract	Versioned generation gateway	src/text_to_sql/generator.py	Bundle + JoinPlan	JSON output and no-call-after-reject tests	JOIN-001	3 days	Member 2
VAL-001	MUST	SQL parser	AST parse and statement allowlist	src/sql_validation/parser.py	CandidateSQL	DDL/DML/multi-statement blocked	GEN-001	2 days	Member 1
VAL-002	MUST	Schema/join validator	Validate objects and JoinPlan	src/sql_validation/schema.py	AST/catalog/JoinPlan	Invalid refs and join mismatch tests	VAL-001,JOIN-001	3 days	Member 1
VAL-003	MUST	Permission validator	Re-check referenced objects	src/sql_validation/policy.py	AST/role/policy	Denied SQL fails	VAL-001,EVID-006	2 days	Member 1
CORR-001	MUST	Bounded correction	Repair diagnosed SQL only	src/self_correction/corrector.py	Validation diagnostics	Max 2 attempts; no-progress and unsafe tests	VAL-001-003	2 days	Both
EXEC-001	MUST	Read-only execution	Execute with timeout/limits	src/execution/postgres.py	Validated SQL	Read-only role and timeout tests	VAL-001-003	2 days	Member 1
AUDIT-001	SHOULD	Request audit trace	Persist lifecycle and versions	src/audit/trace.py	All module outputs	Trace completeness test	Core pipeline	2 days	Member 1
UI-001	MUST	Demo interface	Show full explainable lifecycle	apps/ui/app.py	Core pipeline	Reviewer can run G/C/R demos	Core pipeline	3 days	Member 2
API-001	SHOULD	Query endpoint	Typed POST /query	apps/api/main.py	Contracts/core pipeline	API tests and safe errors	Core pipeline	2 days	Member 1
EXP-001	MUST	Base questions	Create reviewed question set	datasets/base_questions/	Approved schema/evidence	Minimum 60-80 / ~300-400; preferred ~100 / ~500; category balance and review quality	DATA/EVID/QIM	1-2 weeks	Both
EXP-002	MUST	EEVA cases	Create evidence variants and oracles	datasets/eeva_cases/	Base questions	Split by base question; dual review	EXP-001	2 weeks	Both
EXP-003	MUST	B5/B6 experiment	Run primary Olist B5/B6 with identical pipelines except EEVA	experiments/b5_b6.py	Frozen held-out Olist set	Config diff only EEVA; raw outputs saved	EXP-002,core pipeline	1 week	Both
EXP-004	MUST	Final metrics + secondary	Compute final Olist results; freeze rules/prompts; then run separate Base-Lite secondary	experiments/metrics.py; results/	EXP-003 + frozen config	Olist metrics verified; Base-Lite results stored separately; no combined score	EXP-003	3 days	Member 2
TEST-001	MUST	Core test suite	Automate unit/integration/golden	tests/	All modules	Required tests pass	Ongoing	Ongoing	Both
SEC-001	SHOULD	Adversarial safety suite	Prompt/evidence/SQL attacks	tests/security/	Core system	Unsafe requests fail closed	Core pipeline	3 days	Member 1
DEPLOY-001	SHOULD	Docker Compose	Reproducible DB/API/UI	docker-compose.yml	Stable app	Clean-machine smoke test	Core pipeline	2 days	Member 1
DOC-001	MUST	Research integration	Update paper/report with actual results	docs/paper_update.md	EXP-004	No placeholders or overclaim	EXP-004	3-5 days	Both

Integrated Codex additions before EXP-003

- Persist initial EvidenceSnapshot IDs and B6 recovery snapshot IDs.
- Add canonical_obligation_key = database_id + evidence_type + canonical_target_id + constraint_type and offline QIM metrics.
- Add decision_stage and UNSUPPORTED_OPERATION tests.

58. Final Consistency Checklist

Table 49. Final consistency checklist.

Check	Status	Resolution
One project identity	PASS	Official title: DriftSQL: An Enterprise Query Intelligence and Validation System.
One architecture	PASS	QIM -> R(q) -> retrieval/normalization -> EEVA -> decision -> join -> LLM -> Stage 2 -> execution/result; database_id adapters do not change the pipeline.
Olist primary; Base-Lite secondary	PASS WITH VERIFICATION	Olist primary; Base-Lite only after Olist freeze; results separate; Olist source/schema verification still required.
No invented schema	PASS	DATA-001 required before data decisions.
EEVA central and heterogeneous	PASS	Schema, metadata, glossary, docs, relationships/values and policy.
QIM creates R(q)	PASS	Contract and task specified.
RAG feeds EEVA	PASS	No direct bypass to generator.
Stage 1 and Stage 2 separate	PASS	Distinct stages; SQL correction max 2 attempts and cannot change the EEVA evidence decision.
Legacy modules resolved	PASS	AASE/SDARE/MAL/SFO/CARD merged; LCHR stretch; HF removed.
Hard constraints non-compensatory	PASS	Rule engine precedes soft/optional logic.
ML optional	PASS	Stretch only.
CLARIFY bounded and returns to EEVA	PASS	Max 2 cycles; material evidence progress required; no progress/exhaustion -> REJECT / RECOVERY_EXHAUSTED.
PostgreSQL final security authority	PASS	Read-only role and policy re-check.
B5/B6 preserved	PASS	Same downstream pipeline; only EEVA differs.
Leakage prevented	PASS	Group split by base question.
Results not fabricated	PASS	Placeholders prohibited.
Two-student six-month feasibility	PASS WITH SCOPE CONTROL	Tiered targets, bounded loops, one frozen LLM; Base-Lite only in final phase.
PDF readability	PASS	Final changed pages rendered and visually checked for clipping, overlap and broken glyphs.

Final decision: implementation begins with DATA-001 by obtaining/verifying the actual Olist source files and license, profiling the data, and validating the real schema/keys/relationships. No dataset field or implementation completion is asserted before verification. LiveSQLBench Base-Lite is not an early dependency; it is introduced only after the Olist implementation, EEVA rules and prompts are stable/frozen. The Master Blueprint controls research meaning; this implementation blueprint controls task order, engineering controls and acceptance criteria.

IMPLEMENTATION FREEZE: PASS. Section 59 resolves the six pre-implementation specification issues and is authoritative for those topics. Implementation may begin with DATA-001.

59. Implementation Freeze - Resolved Pre-Implementation Specifications

Implementation authority. Section 59 resolves the six pre-implementation ambiguities identified during final review. For these six topics, Section 59 supersedes older incomplete or inconsistent wording. The architecture itself is unchanged.

Issue	Frozen resolution
1. B5 baseline contract	B5 and B6 start from the same INITIAL EvidenceSnapshot . B5 bypasses EEVA and creates a BaselineGenerationBundle. B6 passes the same initial snapshot through EEVA and creates a ValidatedEvidenceBundle only on GENERATE. All downstream model, prompt, join implementation, Stage-2, correction, database and execution settings are identical.
2. R(q) correctness	Create one manually reviewed Gold R(q) per base question for offline evaluation. Gold R(q) is never supplied to runtime. Score canonical obligations and report Mandatory Obligation Recall plus Precision/F1 where practical.
3. Unsupported queries	QIM performs an early read-only scope precheck. Unsupported write/DDL/admin requests return final REJECT with reason UNSUPPORTED_OPERATION at stage QIM_PRECHECK. Generator, correction and executor are not called.
4. Clarification authority	A user may clarify intent, filters or select among approved interpretations, but cannot override authoritative schema, relationships, policy/permissions, provenance, glossary or verified enterprise facts. Clarification always returns through retrieval/normalization and EEVA.
5. Freshness / staleness	Currentness is source-specific: live PostgreSQL catalog for schema/relationships, active DB policy for authorization, current approved/effective versions for glossary/docs, and the current approved metadata snapshot. No arbitrary global time threshold is used.
6. Team ownership	Use the Production Blueprint responsibility matrix as the implementation authority. The research remains jointly owned.

59.1 Baseline execution modes

Shared start: QIM, R(q), direct/semantic retrieval and normalization produce one immutable **INITIAL EvidenceSnapshot** keyed by request_id and database_id. This exact initial snapshot is persisted for the paired B5/B6 experiment.

B5 / eeve_enabled=false: Initial EvidenceSnapshot -> BaselineGenerationBundle -> same join implementation -> generation -> Stage 2 -> bounded correction -> execution. The baseline bundle preserves retrieved evidence after only common format validation and mandatory environment access filtering; it does not run EEVA authority, freshness, conflict, coverage, recoverability or G/C/R rules.

B6 / eeve_enabled=true: Same initial EvidenceSnapshot -> EEVA -> ValidatedEvidenceBundle on GENERATE. CLARIFY may create later recovery EvidenceSnapshots and re-enter EEVA; REJECT stops before the LLM. Recovery snapshots are treatment outputs and do not invalidate the pair.

Required experiment guard: experiments/b5_b6.py must assert that the question, database_id, INITIAL EvidenceSnapshot ID, model/version, prompt, decoding settings, join implementation, Stage-2 rules, correction bound and execution configuration are identical; only eeve_enabled differs initially.

59.2 Gold R(q) contract

Store Gold R(q) only under the offline experiment dataset. There is one Gold R(q) per base question. Suggested fields: question_id, gold_obligation_id, evidence_type, canonical_target_id, constraint_type, mandatory, expected_source_types, reason and annotation_version.

After approved alias normalization, match predicted and gold obligations using **database_id + evidence_type + canonical_target_id + constraint_type**. Report Mandatory Obligation Recall and, where meaningful, obligation Precision/F1. Gold annotations must never be passed into runtime QIM, retrieval, EEVA, generator or clarification controller.

59.3 Unsupported-intent precheck

QIM returns intent=UNSUPPORTED for out-of-scope write/DDL/admin operations. The controller emits FinalResponse(decision="REJECT", reason_code="UNSUPPORTED_OPERATION", decision_stage="QIM_PRECHECK"). No generator, correction or executor call is permitted. EEVA three-class metrics exclude these precheck rejects unless separately analyzed.

59. Implementation Freeze - Runtime Authority Rules

59.4 Clarification authority

Allowed: user clarification may disambiguate intent, filters, requested metric meaning or choose among approved interpretations already exposed by the system. Not allowed: a user answer cannot create or override schema facts, relationship facts, permission/policy facts, provenance, source authority or approved enterprise definitions.

Controller rule: accepted clarification -> update QueryIntent/R(q) -> execute only approved recovery actions -> rebuild/update EvidenceSnapshot -> run EEVA again. Direct clarification -> SQL generation is prohibited. Existing max-two-cycle, progress-hash and RECOVERY_EXHAUSTED behavior remains mandatory.

59.5 Frozen freshness policy

Evidence type	Authoritative current state	When stale/recoverable
Schema / relationships	Active PostgreSQL catalog / information_schema + approved current relationship snapshot.	Cached snapshot/hash differs while live refresh is available.
Authorization	Active PostgreSQL role/policy configuration.	Cached policy differs; permission denial itself remains a hard failure.
Business glossary	Latest approved definition whose effective period applies to the query.	Older/superseded approved definition is retrieved while a current one exists.
Documentation	Latest applicable approved document version by authority and effective date.	Older applicable version is retrieved and can be replaced by the current approved source.
Metadata	Current approved metadata version/snapshot linked to the active schema.	Version/hash no longer matches the approved/current state and refresh is available.

Suggested implementation: keep freshness/currentness logic in a small deterministic component (for example src/evidence/freshness.py), version the policy, and store freshness_policy_version in AuditTrace and ExperimentCase. Freeze rules before held-out evaluation.

59.6 Final implementation ownership

Area	Primary owner	Shared review
PostgreSQL, schema, migrations	Member 1	Member 2 reviews data semantics
Evidence repository, metadata/glossary	Member 1	Both approve contracts and definitions
QIM and AASE	Member 2	Member 1 reviews R(q) schema needs
Retrieval, MAL, SDARE	Member 2 primary	Member 1 reviews authoritative sources
EEVA hard rules and decision engine	Member 1	Both review labels and errors
Conflict detection and clarification	Member 2	Both review recovery behavior
Join reasoning and LLM gateway	Member 2	Member 1 reviews SQL safety interfaces
SQL validation, SFO and executor	Member 1	Member 2 reviews generated cases
Experiments, graphs and UI	Member 2 primary	Both own B5/B6 and results
Paper, report, demo and viva	Shared	Both must explain the complete system

This matrix is the single implementation ownership source. Changes require both members to update the task board and documentation; they do not change research authorship or shared responsibility for B5/B6 results.

59. Implementation Freeze - Codex Start Contract

59.7 Files/tasks to add or adjust

Resolution	Implementation target
B5/B6 paired mode	experiments/b5_b6.py; persist initial EvidenceSnapshot; eeve_enabled config; config-diff assertion; recovery snapshot logging.
Gold R(q)	datasets/gold_obligations/; experiments/qim_metrics.py; canonical_obligation_key; runtime-exclusion test.
Unsupported precheck	src/query_intelligence/parser.py + controller; UNSUPPORTED_OPERATION; decision_stage=QIM_PRECHECK; generator-not-called test.
Clarification authority	src/clarification/controller.py; allowed clarification fields; authoritative-source recheck; no-direct-generation test.
Freshness policy	Versioned deterministic source-specific currentness rules; stale fixtures for schema, metadata, glossary, docs and policy.
Ownership	docs/implementation_ownership.md or task board matching Section 46/59.6.

59.8 Acceptance tests before the first research run

1. Paired B5/B6 run uses the same request, question and INITIAL EvidenceSnapshot ID; only eeve_enabled differs initially. B6 recovery snapshot IDs are logged as treatment outputs.
2. At least the first 20 demo base questions have reviewed Gold R(q) annotations and QIM obligation metrics can be computed without runtime access to gold data.
3. INSERT/UPDATE/DELETE/DDDL requests return UNSUPPORTED_OPERATION at QIM_PRECHECK and prove generator_called=false and execution_called=false.
4. A clarification can select an approved business interpretation but cannot change a permission denial, schema fact, relationship fact or provenance/source-authority rule.
5. A stale schema snapshot is detected by hash/version mismatch and is recoverable when the live catalog is available; current schema passes.
6. The team task board matches the Section 59.6 ownership matrix.

59.9 Definition of ready

With these six decisions frozen, DriftSQL is READY TO START IMPLEMENTATION. Begin with DATA-001 Olist source/license/file/schema verification, followed by repeatable PostgreSQL import, database_id/generic adapter, typed contracts and the evidence layer. Do not delay implementation for further architecture redesign unless verified data or experiments reveal a concrete contradiction.

Codex rule: for the six topics in Section 59, follow Section 59 exactly. For research meaning use the Master Blueprint; for task order, engineering controls and acceptance criteria use this Production Blueprint. Never invent Olist schema facts before DATA-001 verifies them.